كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

Syrian Private University

Faculty of Artificial Intelligence Engineering

An Online Learning Web Application for Courses Management

Software Requirements Specification (SRS)

Graduation Project 2: This research is submitted in partial fulfillment of the requirements for obtaining the Bachelor's Degree in Artificial Intelligence Engineering, Department of Software Engineering and Intelligent Information Systems.

Supervised by:

Dr. Kadan Joumaa

Submitted by:

Yassin Yahya Boushi

Houssam Khaled Al-Ghawi

Academic Year 2026

Table of Contents

List of Tables

List of Figures

List of Abbreviations and Scientific Terms

Table 1. Abbreviations and Scientific Terms

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ERD	Entity Relationship Diagram
FR	Functional Requirement
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JSON Data Interchange Format
LLM	Large Language Model
LMS	Learning Management System
MCQ	Multiple Choice Question
MoSCoW	MoSCoW Prioritization Method
NFR	Non-functional Requirement
RAG	Retrieval-Augmented Generation
REST	REST Architectural Style for Interfaces
SCM	Software Configuration Management
SEO	Search Engine Optimization
SPA	Single Page Application
SRS	Software Requirements Specification
SSR	Server-Side Rendering
UC	Use Case
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

Abstract

Despite the notable development that learning management systems (LMS) have witnessed in recent years, many of them still focus primarily on content management and the presentation of educational materials without providing intelligent capabilities for understanding and interacting with that content. This project aims to develop an Intelligent Learning Management System that employs modern artificial intelligence techniques to transform the traditional learning environment into an interactive one capable of analyzing content and providing intelligent academic support.

The proposed system relies on integrating the Gemini API with Retrieval-Augmented Generation (RAG) technology, whereby educational materials such as documents and course content are converted into a searchable knowledge base using embedding techniques and the ChromaDB vector database. When a student asks a question, relevant information is retrieved from the educational content before being sent to the Gemini model via the API, enabling the generation of answers grounded in the actual course sources, thereby increasing answer accuracy and reducing the production of information unrelated to the educational content.

The system also leverages the Gemini API for the automatic generation of quizzes, whereby multiple-choice and true/false questions are created based on documents uploaded by the instructor, which helps reduce the time and effort required to prepare assessments while maintaining their relevance to the course content.

The project seeks to address a set of challenges facing current e-learning systems, such as the increased effort required from faculty members to prepare exams, weak intelligent interaction with content, and the absence of educational assistance grounded in course sources. To achieve this, the system combines course management, intelligent assessment, and an academic assistant based on RAG technology within a unified educational platform.

This project represents a modern model for intelligent educational platforms, as it contributes to improving the quality of the educational process, reducing the burden on faculty members, and enabling students to access accurate and reliable answers based on course content, in line with modern trends in digital transformation and the application of artificial intelligence in education.

Keywords: Learning Management System; Retrieval-Augmented Generation; Gemini API; ChromaDB; Embeddings; Artificial Intelligence in Education; Automatic Quiz Generation; Next.js; MongoDB.

الملخص

على الرغم من التطور الملحوظ الذي شهدته أنظمة إدارة التعلم (LMS) حديثًا خلال السنوات الأخيرة، إلا أن العديد منها لا يزال يركز بصورة أساسية على إدارة المحتوى وعرض المواد التعليمية دون توفير إمكانات ذكية لفهم المحتوى والتفاعل معه. يهدف هذا المشروع إلى تطوير نظام إدارة تعلم ذكي (Intelligent Learning Management System) يوظف تقنيات الذكاء الاصطناعي الحديثة لتحويل بيئة التعلم التقليدية إلى بيئة تفاعلية قادرة على تحليل المحتوى وتقديم دعم أكاديمي ذكي.

يعتمد النظام المقترح على دمج واجهة برمجة تطبيقات Gemini API مع تقنية التوليد المعزز بالاسترجاع (Retrieval-Augmented Generation - RAG)، حيث تُحوَّل المواد التعليمية المتمثلة بالوثائق والمحتوى الدراسي إلى قاعدة معرفية قابلة للبحث الدلالي باستخدام تقنيات التضمين (Embeddings) وقاعدة بيانات متجهية ChromaDB. وعند طرح الطالب سؤالاً، يتم استرجاع معلومات ذات صلة من المحتوى التعليمي قبل إرسالها إلى نموذج Gemini عبر واجهة البرمجة، مما يسمح بتوليد إجابات تستند إلى مصادر المقرر الفعلية، ويزيد من دقة الإجابات ويحد من إنتاج معلومات غير مرتبطة بالمحتوى التعليمي.

كما يستفيد النظام من Gemini API في التوليد الآلي للاختبارات، حيث يتم إنشاء أسئلة الاختيار من متعدد وأسئلة الصح والخطأ اعتمادًا على الوثائق التي يرفعها المدرس، مما يساهم في تقليل الوقت والجهد اللازمين لإعداد التقييمات مع المحافظة على ارتباطها بالمحتوى الدراسي.

يسعى المشروع إلى معالجة مجموعة من التحديات التي تواجه أنظمة التعلم الإلكتروني الحالية، مثل زيادة الجهد المطلوب من أعضاء هيئة التدريس في إعداد الاختبارات، وضعف التفاعل الذكي مع المحتوى، وغياب المساعدة التعليمية المعتمدة على مصادر المقرر. ولتحقيق ذلك، يجمع النظام بين إدارة المقررات، والتقييم الذكي، والمساعد الأكاديمي المعتمد على تقنية RAG ضمن منصة تعليمية موحدة.

ويمثل هذا المشروع نموذجًا حديثًا لمنصات التعليم الذكية، إذ يسهم في تحسين جودة العملية التعليمية، وتقليل العبء على أعضاء هيئة التدريس، وتمكين الطلاب من الوصول إلى إجابات دقيقة وموثوقة مستندة إلى محتوى المقرر، بما ينسجم مع الاتجاهات الحديثة في التحول الرقمي وتطبيقات الذكاء الاصطناعي في التعليم.

الكلمات المفتاحية: نظام إدارة التعلم؛ التوليد المعزز بالاسترجاع؛ Gemini API؛ ChromaDB؛ التضمين؛ الذكاء الاصطناعي في التعليم؛ التوليد الآلي للاختبارات؛ Next.js؛ MongoDB.

Chapter 1. Introduction

1.1 Overview and Background

The continuous expansion of digital infrastructure has reshaped the way higher-education institutions deliver knowledge and manage their academic activities. Learning Management Systems (LMS) have become central platforms that unify course delivery, assessment, communication, and administrative oversight within a single web environment, reducing reliance on manual procedures and enabling scalable, asynchronous learning. The rapid advancement of generative Artificial Intelligence services, such as AI APIs, has expanded the role of Learning Management Systems, which are no longer limited to storing and distributing content but are now capable of interpreting content, answering learner inquiries, and generating assessment materials based on educational content.

The system presented in this thesis is a modern Learning Management System implemented using the Next.js framework, following a hybrid architecture that combines the interactivity of a Single Page Application (SPA) with the indexability and improved initial-load performance of Server-Side Rendering (SSR), backed by a document-oriented storage layer based on MongoDB accessed through a secured RESTful API protected using NextAuth.js. The system supports three user roles: Student, Instructor, and Administrator, and covers registration, authentication, profile management, browsing the course catalog, enrollment, delivery of educational content, quizzes, certificate issuance, evaluations, and centralized management of users and content.

The final implemented version adds four new capabilities to this foundation. Two of these are Artificial Intelligence features: (1) the Intelligent Student Assistant, based on Retrieval-Augmented Generation (RAG), which uses the Gemini API to generate answers grounded in educational documents retrieved from the knowledge base, and (2) Automated Quiz Generation, in which the platform uses the Gemini API to automatically create multiple-choice and true/false questions based on the educational materials uploaded by the instructor. The other two features are: (3) the Instructor Analytics Dashboard, which displays charts of student progress, course performance, and assessment statistics, and (4) the Administrator Analytics Dashboard, which displays statistics on users, courses, enrollments, and posts.

1.2 Problem Statement and Significance

Despite the widespread adoption of e-learning systems and platforms, a range of technical and functional challenges continue to limit their effectiveness and efficiency. At the technical level, some educational platforms rely heavily on client-side processing, which can limit the benefits of Server-Side Rendering (SSR) and negatively affect the discoverability and indexing of course pages by search engines. This, in turn, affects the visibility and accessibility of public educational content through search results. Some solutions also suffer from a fragmented user experience across different roles, weak separation between public informational pages and authenticated learning areas, as well as administrative procedures that can be complex or inflexible when managing users, roles, courses, and assessments.

On the other hand, some e-learning environments focus more heavily on providing services and features oriented toward students, such as content access, taking quizzes, and tracking progress, while instructors do not always receive a comparable level of intelligent and analytical tools to help them perform their teaching tasks and make decisions related to courses and students. This challenge is particularly evident in preparing assessments, tracking student performance, and analyzing student activity, where these processes may rely on manual effort or on limited reports that do not provide a comprehensive picture of course performance. There is therefore a need for an environment that does not focus solely on improving the student experience but also supports the instructor through tools that help in preparing assessments and analyzing the performance of students and courses.

A number of additional functional and educational gaps contributed to shaping the development of the final version of the system. First, learners who need additional clarification on course material often have to leave the platform and rely on general-purpose chatbots that are not directly linked to the course content, which can produce answers that appear correct and natural but are not necessarily grounded in the course's own educational source. The proposed system addresses this challenge through the Intelligent Assistant, which grounds its generated answers in the course's educational content. Second, preparing appropriate educational assessment questions requires considerable time and effort from instructors, particularly when dealing with a large number of courses and materials. The system supports instructors by generating quiz questions based on educational content, with the ability to review and edit them before approval. Third, instructors

and administrators in some e-learning environments lack an integrated and actionable visual representation of data related to activity, interaction, and performance, making it more difficult to track students and courses and to make data-driven decisions. This highlights the importance of providing dedicated analytics dashboards for both the instructor and the administrator, presenting information tailored to the nature of each role and its associated responsibilities.

The importance of addressing these challenges stems from the increasingly central role that e-learning platforms play in supporting the educational process. The development of modern learning systems should not be limited to improving the student experience alone; it must also equip instructors and administrators with the tools needed to manage the educational process and monitor its outcomes. A well-integrated system can combine technical performance and content discoverability through search engines, provide an interactive learning experience for students, support instructors in preparing assessments and analyzing student and course performance, and provide comprehensive analytics for administrators. In this way, the proposed system contributes to building a more integrated, interactive, and data-driven digital learning environment, achieving a better balance among the needs of students, instructors, and administrators.

1.3 Project Objectives (SMART)

1.3.1 Core Project Idea

The core idea of this project is to redefine the role of traditional Learning Management Systems, transforming them from platforms limited to managing educational content into intelligent educational platforms capable of understanding, analyzing, and interacting with content using modern Artificial Intelligence techniques. The project achieves this by integrating the Gemini API with Retrieval-Augmented Generation (RAG) technology within a unified educational environment, ensuring the delivery of more accurate, effective, and reliable educational services.

The system relies on converting unstructured educational content, such as lecture files and documents, into a searchable knowledge base that supports semantic retrieval using Embeddings and the ChromaDB vector database. When a student submits a query, the system retrieves the passages most relevant to the educational content and then sends them to the Gemini API to generate an answer grounded in the course content, thereby reducing the phenomenon of

inaccurate information (hallucination) and maintaining a direct link to the approved educational source.

The system also uses the Gemini API to automatically generate quiz questions, creating multiple-choice and true/false questions based on the documents uploaded by the instructor, with the ability to review and edit them before approval.

The project aims to address a range of challenges facing current e-learning systems, such as the considerable time required to prepare quizzes, weak interaction with content, and the inability of traditional systems to provide intelligent, personalized academic support for each student. To achieve this, the system leverages Embeddings, the ChromaDB vector database, Semantic Retrieval, and the Gemini API to provide an interactive educational environment that combines course management, intelligent assessment, and educational assistance grounded in the actual knowledge of the course.

Accordingly, the project is not merely the development of an e-learning management system, but rather an integrated model of an intelligent educational platform based on the latest generative Artificial Intelligence technologies, contributing to improving the quality of the educational process, reducing the burden on faculty members, and enabling students to access accurate and reliable information, in line with modern trends in digital transformation and smart education.

The project objectives were formulated according to SMART criteria, so that they are Specific, Measurable, Achievable, Relevant, and Time-bound, as follows:

- Design and implement an integrated Learning Management System using Next.js with a hybrid architecture combining SPA and SSR, and deliver a production-ready version within the project's time frame.
- Provide the core academic functions, including courses, enrollment, content, quizzes, certificates, evaluations, and user management.
- Develop an Intelligent Student Assistant using RAG to answer inquiries based on educational content linked to the courses.
- Develop an Automated Quiz Generation feature using Gemini AI to create multiple-choice and true/false questions based on the materials uploaded by the instructor.

- Provide the instructor and administrator with dashboards to view data on students, courses, evaluations, and platform usage, and verify the accuracy of the results against the database.
- Improve the visibility of public pages in search engines through SSR, semantic HTML, and appropriate metadata, with the ability to measure this using SEO auditing tools.
- Document the system development process in accordance with IEEE 830 and ISO/IEC/IEEE 29148 standards, and produce the diagrams and software models needed to support future evaluation and maintenance.

1.4 Scope, Constraints, and Assumptions

The scope of the project includes the analysis, design, implementation, testing, and documentation of a web-based Learning Management System supporting three main roles: Student, Instructor, and Administrator, with clear separation of permissions, interfaces, and role-based access verification, along with the four features added in the final version. The system is built on Next.js with a hybrid architecture and communicates with a backend RESTful API over HTTPS, with data stored in MongoDB, while external Artificial Intelligence services are used without internally training any language models.

Assumptions: The project assumes the availability of a modern web browser and a stable internet connection, that instructors upload supported educational content of adequate textual quality, that administrators possess the basic knowledge required to manage the platform, and that a suitable hosting environment is available in terms of availability and security.

Constraints: The four new features were developed within a summer term of only two months, which imposed a limited time frame for implementing and testing the Artificial Intelligence and analytics functions and ensuring their integration with the existing system. The performance of the Artificial Intelligence functions also depends on external services and models and their availability, as well as on the quality of the uploaded educational content. Given the academic and simulated nature of the project, a Mock Payment Gateway was implemented instead of integrating a real payment gateway, and therefore the system does not include actual processing of financial payments. Likewise, the system is limited to a web application and does not include native Android or iOS applications, nor does it include internal training or fine-tuning of language models,

automatic grading of essay-type answers, prediction of student performance, or identification of at-risk students.

Table 2. Project Scope, Constraints, and Assumptions

Aspect	In Scope	Out of Scope
User Roles	Student, Instructor, Administrator	Additional roles such as parents/guardians and academic supervisors
Educational Content	Courses, units, chapters, lessons, and documents	Live lectures and synchronous virtual classrooms
Assessment	Quiz creation and generation of multiple-choice and true/false questions	Automatic grading of essay and free-text answers
Artificial Intelligence	Gemini-based student assistant and RAG-based quiz generation	Internal training or fine-tuning of language models
Analytics	Instructor and Administrator analytics dashboards	Prediction of grades or identification of at-risk students
Payment	Simulated payment process for academic purposes and workflow testing	Real payment processing and integration with actual payment gateways
Deployment	Web application for modern browsers	Native Android and iOS applications
Time Frame	Development of the four features within a two-month summer term	Development and expansion of additional features beyond the project time frame

1.5 Research Methodology

The project adopted the Engineering Methodology as the most suitable approach for its nature and objectives, as the project focuses on analyzing a real-world problem, defining system requirements, then designing and implementing an integrated software solution and verifying its compliance with the defined requirements. This methodology relies on the systematic application of software engineering principles and methods, starting with studying and analyzing the problem, proceeding through system design and implementation, and concluding with testing and evaluation.

The Engineering Methodology was applied by translating system needs into traceable, documentable functional and non-functional requirements, followed by modeling the system and designing its architecture, database, and interfaces, then implementing the system's components and functions incrementally. The development process also included integrating the required intelligent features and verifying their integration with the rest of the system's components.

In the final stage, the system was verified by executing test cases and scenarios linked to the system requirements and the different user roles, and analyzing the results to verify the correctness of the functions and the extent to which the defined objectives were achieved. The Engineering Methodology thus provided an integrated and organized framework for moving from problem and requirement definition to building, testing, and evaluating the system.

1.6 Thesis Structure

The thesis consists of eight interrelated chapters, each addressing a specific aspect of the system development stages, starting with defining the problem and objectives, proceeding through analysis, design, and implementation, and concluding with testing, evaluation, and the presentation of results and conclusions. The organization of the chapters was designed to achieve a logical sequence that ensures continuous traceability from system requirements to design, implementation, testing, and evaluation.

1.6.1 Chapter One — Introduction

Defines the problem the system seeks to address, and clarifies the project's objectives, significance, scope, and associated constraints and assumptions, in addition to the expected contribution and the research methodology followed.

1.6.2 Chapter Two — Development Methodology and Project Management

This chapter presents the methodology adopted for developing the system, the reasons for its selection, and its suitability to the nature of the project. It also covers the project timeline, development phases and key milestones, risk management, the distribution of roles and responsibilities among team members, quality assurance mechanisms, project management tools,

continuous monitoring and evaluation mechanisms, as well as version and software configuration management.

1.6.3 Chapter Three — Theoretical Framework, Prior Studies, and Related Work

This chapter addresses the theoretical and scientific background related to the project's field, reviews prior studies, research, and similar systems, and analyzes the most prominent modern technologies and frameworks used in the field. It also presents a comparison of existing systems in terms of features and capabilities, identifies the gap addressed by the project, clarifies the innovative aspects and proposed solutions, and discusses relevant modern and future trends.

1.6.4 Chapter Four — Requirements Analysis and System Modeling

This chapter focuses on identifying and analyzing the system requirements, starting with the technical, economic, and operational feasibility study, and presents the functional and non-functional requirements and their prioritization using the MoSCoW methodology, in addition to modeling the system using UML diagrams, documenting use cases, analyzing requirements, and building a requirements traceability matrix linking them to the design and testing phases.

1.6.5 Chapter Five — Architectural and Detailed System Design

This chapter explains how the system was designed and how the requirements were translated into an implementable technical structure. It includes the high-level architectural design, the system's components and their interactions, the adopted architectural design pattern, the database design, the entity-relationship model, the relational schema, and the associated constraints. It also covers the design of the APIs, the authentication and authorization mechanisms, and the design of security, permissions, and data protection.

1.6.6 Chapter Six — Development Environment and Software Implementation

This chapter documents the technical environment used to build the system, including the programming languages, frameworks, libraries, tools, and their versions, the development

environment and project structure, the database server, and dependency, version, and source control management. It also presents the implementation details of the main modules, the APIs, the data access and business logic layers, and the user interfaces, in addition to the implementation of the intelligent features and analytical components included in the system.

1.6.7 Chapter Seven — Testing and Evaluation

This chapter aims to verify that the system operates in accordance with the defined requirements and specifications by presenting the test plan, responsibilities, and testing tools, and the types of tests performed, including unit, integration, system, acceptance, performance, usability, compatibility, and regression testing where needed. It also presents the test results and an analysis of passed and failed cases and remaining defects, alongside quality metrics and an evaluation of the system's usability and user feedback.

1.6.8 Chapter Eight — Results, Analysis, and Conclusion

This chapter presents the actual results obtained after implementing and testing the system, including system outputs, statistics, charts, and performance indicators. It also includes a comparative analysis with existing systems or solutions, an evaluation of the extent to which the objectives defined in Chapter One were achieved, an analysis of the system's strengths and weaknesses, and a discussion of Key Performance Indicators (KPIs). The chapter concludes by presenting the main achievements and contributions, the challenges and constraints faced during the project, along with recommendations and proposals for future development and possible improvements in subsequent versions.

1.7 Definitions

Table 3. Definition of Learning Management System Terms

Term	Definition
LMS	Learning Management System: a web-based platform that organizes the delivery, management, and assessment of educational courses for students, instructors, and administrators.
SPA	Single Page Application: a web application that loads a single HTML document and dynamically updates its content through client-side rendering without a full page reload.
SSR	Server-Side Rendering: rendering pages on the server before sending fully formed HTML to the client, improving initial load performance and search engine indexability.
SEO	Search Engine Optimization: the set of practices used to improve the visibility and ranking of web content in search engine results.
REST API	Representational State Transfer API: an architectural pattern for designing network services using stateless HTTP requests and resource-oriented endpoints.
Next.js	A production-grade React-based web framework that supports hybrid rendering patterns, including client-side rendering, server-side rendering, and statically generated pages.
MongoDB	A document-oriented NoSQL database that stores flexible, loosely schema-constrained records, used as the system's storage layer.
RAG	Retrieval-Augmented Generation: an architecture in which relevant passages are first retrieved from a document collection and then provided to a language model, so that the generated answers are grounded in those sources.
Intelligent Student Assistant	The Artificial Intelligence feature that answers student questions about course material using Retrieval-Augmented Generation constrained to the educational documents uploaded to the platform.

Table 4. Definition of Automated Quiz Generation Terms

Term	Definition
Automated Quiz Generation	The Artificial Intelligence feature in which Gemini AI automatically generates multiple-choice and true/false questions from the educational materials uploaded by the instructor.

Term	Definition
Gemini AI	The external generative Artificial Intelligence model service used by the system to generate assessment questions from uploaded educational content.
MCQ	An assessment item that presents several answer options, one of which is correct.
Analytics Dashboard	A reporting interface that aggregates platform data and displays it visually through charts and statistics; in this system it is a software analytics feature and not an Artificial Intelligence feature.
Quiz	A structured assessment consisting of one or more questions used to measure a learner's understanding of a topic within a course.
Certificate	An official document issued by the system to a learner upon successfully completing the requirements of a course.
Enrollment	The process by which a registered student is linked to a course, thereby being granted access to its educational materials and assessments.
Role	A named set of permissions that defines the actions a user is authorized to perform within the system.
Administrator	A user with extended privileges responsible for managing accounts, roles, courses, and the platform's general configuration.
Instructor	A user authorized to create and manage courses, publish educational materials, upload documents, and evaluate student performance.
Student	A user authorized to browse the course catalog, enroll in courses, study content, use the Intelligent Assistant, take quizzes, and obtain certificates.

Chapter 2. Development Methodology and Project Management

2.1 Adopted Development Methodology

The project adopted the Incremental Development methodology as the software development model used to build the system. This methodology is based on developing the system through a series of successive increments, such that each increment represents an interrelated set of functionalities that pass through a complete development cycle including analysis, design, implementation, and testing before being integrated into the overall system version. With this approach, the output of each phase is not limited to design or analysis documents, but rather consists of executable and testable software functionalities, enabling early verification of system behavior and continuous evaluation of its outputs.

Incremental Development was selected due to its suitability to the nature of the project, particularly since the system consists of a set of functions and components that can be developed and verified progressively. Furthermore, developing the project within a limited time frame requires dividing the scope of work into clear parts that can be implemented and tested sequentially, with the ability to address problems discovered in each increment before moving on to subsequent functionalities.

This methodology provides a number of advantages consistent with the project requirements. It allows for the incremental delivery of functionalities, supports early detection of errors and integration issues, and enables continuous feedback on implemented functionalities and the introduction of necessary improvements before completing the remaining system components. It also helps reduce development risk by avoiding the postponement of verification and testing until the end of the project, instead performing them in parallel with the implementation of each increment.

These advantages are of even greater importance for artificial-intelligence-based features, such as the intelligent assistant based on Retrieval-Augmented Generation (RAG) and the quiz generator, since these functions require verifying the quality of actual outputs during development and making improvements based on results and observations. The incremental approach allows these

features to be developed and tested relatively independently before being integrated into the overall system.

The chosen methodology is also suited to the size of the project team and the nature of the academic work, as it provides a clear structure for dividing tasks and tracking progress without requiring the formal roles and meetings associated with frameworks such as Scrum. At the same time, it allows the team to benefit from certain agile practices, such as sequential development, continuous review, and re-prioritization when needed, without considering the project a formal implementation of the Agile or Scrum framework.

Accordingly, Incremental Development was adopted as the most suitable methodology for the project, owing to its ability to achieve a balance between the organization and documentation required in a graduation project and the flexibility needed to develop and verify system functionalities progressively within the available time frame. The table below presents a methodological comparison between Incremental Development and some common alternatives, indicating the suitability of each to the nature of the project.

2.2 Project Schedule and Planning

2.2.1 Gantt Chart

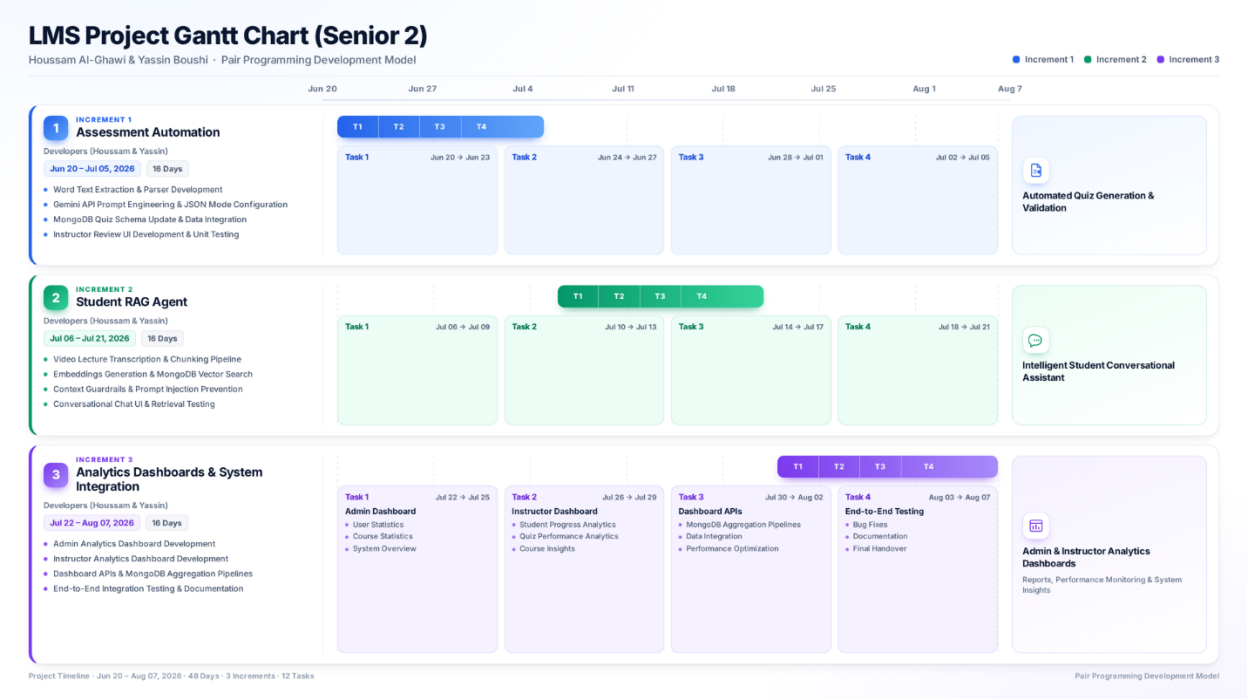


Figure 1. Project Gantt Chart

Table 5. Project Gantt Chart

Increment	ID	Task	Timeframe	Development Phase	Details and Activities	Milestone
Increment 1 (Assessment Automation)	T1	Word Text Extraction & Parser Development	Jun 20 -- Jun 23	Analysis and Implementation	Word Text Extraction & Parser Development	Automated Quiz Generation & Validation
Increment 1 (Assessment Automation)	T2	Gemini API Prompt Engineering & Configuration	Jun 24 -- Jun 27	Design and Implementation	Gemini API Prompt Engineering & JSON Mode Configuration	Automated Quiz Generation & Validation
Increment 1 (Assessment Automation)	T3	MongoDB Schema Update & Data Integration	Jun 28 -- Jul 01	Design and Implementation	MongoDB Quiz Schema Update & Data Integration	Automated Quiz Generation & Validation
Increment 1 (Assessment Automation)	T4	Instructor Review Interface & Unit Testing	Jul 02 -- Jul 05	Interfaces and Testing	Instructor Review UI Development & Unit Testing	Automated Quiz Generation & Validation

Increment	ID	Task	Timeframe	Development Phase	Details and Activities	Milestone
Increment 2 (Student RAG Agent)	T1	Video Lecture Transcription and Chunking	Jul 06 -- Jul 09	Analysis and Implementation	Video Lecture Transcription & Chunking Pipeline	Intelligent Student Conversational Assistant
Increment 2 (Student RAG Agent)	T2	Embedding Generation and Vector Search	Jul 10 -- Jul 13	Design and Implementation	Embeddings Generation & MongoDB Vector Search	Intelligent Student Conversational Assistant
Increment 2 (Student RAG Agent)	T3	Context Constraints and Injection Protection	Jul 14 -- Jul 17	Design and Security	Context Guardrails & Prompt Injection Prevention	Intelligent Student Conversational Assistant
Increment 2 (Student RAG Agent)	T4	Chat Interface and Retrieval Testing	Jul 18 -- Jul 21	Interfaces and Testing	Conversational Chat UI & Retrieval Testing	Intelligent Student Conversational Assistant
Increment 3 (Analytics Dashboards & Integration)	T1	Admin Dashboard	Jul 22 -- Jul 25	Design and Implementation	User statistics, course statistics, and system overview	Admin & Instructor Analytics Dashboards & Final Handover
Increment 3 (Analytics Dashboards & Integration)	T2	Instructor Dashboard	Jul 26 -- Jul 29	Design and Implementation	Student progress analytics, quiz performance analytics, and course insights	Admin & Instructor Analytics Dashboards & Final Handover
Increment 3 (Analytics Dashboards & Integration)	T3	Dashboard APIs	Jul 30 -- Aug 02	Implementation and Optimization	MongoDB aggregation pipelines, data integration, and performance optimization	Admin & Instructor Analytics Dashboards & Final Handover
Increment 3 (Analytics Dashboards & Integration)	T4	Comprehensive Testing and Final Delivery	Aug 03 -- Aug 07	Testing and Delivery	Bug Fixes, Documentation, and final delivery	Admin & Instructor Analytics Dashboards & Final Handover

2.3 Risk Management

Table 6. Risk Management: AI Accuracy Issues

Risk	Probability	Impact	Mitigation Plan
Insufficient accuracy of AI-generated quiz questions or assistant responses	High	High	Manually review a sample of outputs in each batch; Restrict the intelligent assistant's answers to retrieved passages and require it to abstain from answering when retrieval confidence is low; Require instructor approval before publishing any generated quiz; Restrict question generation strictly to materials uploaded by the instructor
External model API limitations (latency, service interruptions)	Medium	High	Apply a timeout and retry policy with safe degradation to manual quiz authoring upon failure; Batch generation requests and cache outputs

Table 7. Mitigation of Educational Material Shortage Risks

Risk	Probability	Impact	Mitigation Plan
Insufficient or poor-quality educational materials for retrieval	Medium	Medium	Monitor quota consumption during testing; Provide instructors with a guideline booklet on acceptable document formats and standards; Define a minimum amount of educational

Risk	Probability	Impact	Mitigation Plan
			content before enabling the assistant for a course; Automatically verify that text can be extracted from uploaded documents
Integration failures between the platform and new existing services	Medium	High	Maintain alternative workflows to ensure the core system's operation is unaffected if an intelligent service fails; Conduct integration testing at the end of each work batch; Define API contracts before beginning implementation
Project schedule overrun	Medium	High	Track weekly progress and compare it against the Gantt Chart baseline; Prioritize and sequence work batches based on priority and dependency to ensure a deliverable system is available after the first batch
Discovery of software defects in late stages	Medium	Medium	Mandatory code review before merging

Table 8. Critical Software Defect Mitigation Plan

Risk	Probability	Impact	Mitigation Plan
Critical software defects	Medium	High	Document and log all defects in a centralized tracking list; Maintain comprehensive unit and integration test coverage for critical paths

Risk	Probability	Impact	Mitigation Plan
Source code conflicts during parallel development	High	Low	Carefully review pull requests before approval; Continuous and frequent synchronization with the main integration branch; Adopt short-lived feature branches
Loss of source code or data	Low	High	Host the source code project on the GitHub platform; Perform periodic, automated database backups

2.4 Team Composition and Task and Role Distribution

This project was developed using Pair Programming and direct collaborative work between the team members (Yaseen Yahya Boushi and Hussam Khaled Al-Ghawi). We worked together in an integrated manner across all phases of the system development lifecycle, starting from requirements analysis and architectural design, through code writing and platform testing, to final delivery and preparation of the final documentation.

To cover all the technical and organizational requirements of the project, we divided the leadership and specialized roles between us as follows:

- Project leadership, coordination, and front-end development: We (led by Hussam Khaled Al-Ghawi with participation from Yaseen Yahya Boushi) managed and coordinated the team's work, maintained the Gantt Chart schedule, and coordinated with the academic supervisor. We also designed and implemented the interactive user interfaces for the three roles (Administrator, Instructor, Student), built the data visualization components in the analytics dashboards, and developed the review interfaces for generated questions.
- Algorithm, artificial intelligence, and database development: We (led by Yaseen Yahya Boushi with participation from Hussam Khaled Al-Ghawi) designed the backend architecture and API layer, designed the MongoDB database schema, indexing, and aggregation pipeline queries, and managed the authentication and security systems. In

addition, we developed the artificial intelligence algorithms and prompt engineering for the Gemini API, and implemented the RAG pipeline and text extraction to build the intelligent assistant.

- Quality assurance and system testing: We worked jointly on planning and executing unit and integration test cases and on tracking and fixing software defects at the end of each increment.
- Source code management and documentation: We adopted daily code synchronization via the GitHub platform and cross code review, in addition to drafting and formatting the project's final academic and technical documentation.

Architectural note: We adopted the principle of "documented shared decisions," whereby all decisions with system-wide impact were subject to bilateral agreement, such that no member could modify any API or database schema relied upon by the other member without prior joint review and approval.

2.5 Project Quality Management

The project adopted an integrated quality management approach applied throughout the development lifecycle, rather than relying on final inspection at project delivery. Platform quality was achieved through the following engineering practices:

- Code Reviews: We adopted a policy of not merging any feature branch into the main integration branch except after careful review by the other member. The review included verifying the correctness of logic, adherence to naming and structural standards, and error handling.
- Automated Integration Testing:
- Unit Testing: We wrote and executed tests covering the functions responsible for core business rules (such as calculating student progress, automatic grading, permission control, and analytics aggregation queries) using representative inputs and edge cases.
- Integration Testing: At the end of each increment, we ran and tested the new software modules together with previously delivered parts of the system to ensure the integrity of existing functionality and the absence of any regression.

- Linting & Static Code Analysis Tools: We used static analysis tools to automatically inspect the source code before merging, which helped detect software errors early, enforce a unified coding style, and ensure modular decomposition and centralized error handling to enhance readability and maintainability.
- Acceptance Criteria: Each increment was verified against predefined acceptance criteria established during the analysis phase. User Acceptance Testing scenarios were executed from the perspective of the platform's three roles (Student, Instructor, and Administrator).
- Continuous Improvement: We analyzed software defects and feedback derived from code reviews to identify their root causes, and applied the resulting solutions and recommendations as binding rules and conventions in subsequent increments.

2.6 Project Management Tools

Table 9. Project Management Tools and Purposes

Tool	Purpose
GitHub Projects	Task management and status tracking (in progress, completed, under review) with direct linkage to the source code
Notion	Documentation of functional and non-functional requirements, API documentation writing, and technical note-keeping
Google Sheets	Preparation of the initial phase schedule and tracking of deadlines and progress
WhatsApp	Fast daily communication between team members to resolve urgent technical issues and direct coordination

2.7 Continuous Monitoring and Evaluation Mechanisms

The project adopted periodic and specific mechanisms to continuously monitor and evaluate progress, consisting of:

- Weekly meetings: We held periodic meetings to review accomplishments, overcome obstacles, and set priorities for the following week.
- Academic supervisor reviews: We presented the outputs of each increment to the academic supervisor and documented their guidance for implementation in subsequent work.

- Progress reports: We compared actual progress against the Gantt Chart baseline at the end of each increment to detect and address any schedule deviation early.
- Burndown Charts: We tracked remaining tasks against time within each increment to visually monitor the completion rate and ensure adherence to deadlines.
- Feedback collection: We conducted test sessions with a sample of students to try out the intelligent assistant and take quizzes, converting their feedback into actual improvements.
- Performance indicator evaluation: At the end of each increment, we reviewed indicators of schedule adherence, test pass rate, open defects, and system response speed.

2.8 Software Configuration Management (SCM)

The Git version control system, hosted on the GitHub platform, was adopted to manage the project's source code. A well-defined strategy was applied to manage the code and facilitate collaborative work among team members:

- Main Branch: Contains exclusively the stable, tested code that is ready for delivery. No modification is merged into this branch except after passing all tests and receiving review approval.
- Develop Branch: The primary branch for daily work, into which modifications are merged after their quality is verified, and which is later merged into the main branch.
- Feature Branches: Each team member creates an independent branch for each task they are working on (e.g., feat-auth for the authentication system, and feat-ui-dashboard for the dashboard), ensuring that modifications do not conflict during parallel development.

2.8.1 Workflow

The workflow begins with creating a new feature branch from the develop branch, then implementing the required changes with clear and descriptive commit messages attached. The branch is then pushed and a pull request is created, which is reviewed by the other member before being merged into the develop branch. Upon completion of each increment and after passing integration testing, the develop branch is merged into the main branch.

Chapter 3. Theoretical Framework and Related Work

3.1 Related Studies in Intelligent Educational Systems

Five recent studies published between 2024 and 2025 were selected for their direct relevance to the project's main themes, namely: intelligent educational assistants based on Retrieval-Augmented Generation (RAG), automatic quiz generation using artificial intelligence models, and learning analytics dashboards. The selection criteria required that the studies be published in peer-reviewed conferences or journals and provide experimental results that could be leveraged in building the proposed system.

3.1.1 Intelligent Educational Assistants Based on RAG

Alario-Hoyos et al. (2024) presented a study titled "Tailoring Your Code Companion: Leveraging LLMs and RAG to Develop a Chatbot to Support Students in a Programming Course," which aimed to develop an intelligent assistant based on RAG technology to answer programming course students' inquiries using the actual course content rather than the language model's general knowledge. The study relied on building a knowledge base from course files and linking it to a large language model through a semantic retrieval mechanism, after which the system was evaluated with a group of students. The results showed that using RAG technology improved the accuracy of answers and their relevance to the educational content, and helped students better understand programming concepts. However, the system was limited to a single course and was not integrated with a comprehensive learning management system or with student performance analysis tools.

The study "Battling Botpoop using GenAI for Higher Education: A Study of a Retrieval-Augmented Generation Chatbot's Impact on Learning" (2024) evaluated the impact of a RAG-based chatbot in a higher-education environment. The researchers built a knowledge base from educational documents and linked it to a generative artificial intelligence model to answer student questions, then measured the quality of the answers and user satisfaction. The results showed that relying on retrieved documents reduced hallucination and improved the reliability of answers compared to general language models; however, the study was limited to the intelligent assistant alone and did not include other functions such as course management, assessments, or analytics.

3.1.2 Automatic Quiz Generation

Kazemitabaar et al. (2024) presented a study titled "CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs," which aimed to evaluate the use of a large-language-model-based intelligent assistant within a university programming course. The study analyzed thousands of interactions between students and the intelligent assistant, in addition to surveys and interviews with students and instructors. The results showed an improvement in the learning experience and increased student satisfaction, and confirmed the importance of designing the assistant to guide students toward critical thinking rather than providing direct solutions. However, the study relied on the language model's general knowledge and did not use RAG technology or the actual course content, nor did it provide tools for quiz generation or student performance analysis.

In the assessment domain, the study "Transformer and Large Language Models for Automatic Multiple-Choice Question Generation: A Systematic Literature Review," published in IEEE Access in 2025, reviewed the latest methods used in generating multiple-choice questions using large language models. The study surveyed a large number of research works, comparing the quality of generated questions and their evaluation methods. The results showed that modern models are capable of producing accurate, content-relevant questions; however, the quality of incorrect distractors still needs improvement, and the study recommended that questions be reviewed by instructors before being adopted. One of the most notable limitations is that most of the systems reviewed in the study were standalone research prototypes that were not integrated within comprehensive learning management systems.

3.1.3 Learning Analytics

Schwendimann et al. (2024) conducted a study titled "Learning Analytics Dashboards Are Increasingly Becoming About Learning and Not Just Analytics: A Systematic Review," aimed at analyzing the evolution of learning analytics dashboards in modern educational systems. The researchers conducted a systematic review of published studies, classifying analytics dashboards according to their educational objectives and data-presentation mechanisms. The results showed that analytics dashboards have come to support decision-making and improve the learning process; however, most systems still focus on displaying indicators and statistics without providing

intelligent interpretations or recommendations to help instructors make educational decisions, and most of them are not integrated with generative artificial intelligence technologies or intelligent educational assistants.

3.1.4 Research Gap

The reviewed studies show that most research has focused on addressing only a single aspect of intelligent learning systems; some studies focused on RAG-based educational assistants, while others focused on automatic quiz generation or on learning analytics dashboards separately. It was also found that the majority of platforms and research efforts focused primarily on improving the student experience and engaging learners, while neglecting the instructor's needs, as most systems were limited to providing administrative tools or simple statistical reports without offering intelligent tools to help instructors prepare quizzes, analyze student performance, monitor course progress, and make data-driven educational decisions.

Building on these gaps, the current project aims to provide an integrated intelligent learning management system that combines an intelligent RAG-based assistant using the Gemini API, automatic quiz generation, an advanced analytics dashboard for instructors, and an analytics dashboard for administrators within a single platform, achieving a balance between the needs of students, instructors, and administrators, and providing a more integrated and effective educational environment compared to the solutions addressed in the reviewed studies.

3.2 Modern Technologies and Frameworks in the Field

The development of modern intelligent learning management systems relies on a set of software frameworks and technologies that contribute to improving performance, scalability, and user experience, in addition to effectively integrating artificial intelligence technologies. This project relies on the following set of modern technologies:

- Next.js

Next.js is a modern framework built on React for developing advanced web applications, characterized by its support for Server-Side Rendering (SSR) alongside Client-Side Rendering (CSR) and Static Site Generation (SSG). SSR contributes to improving initial page load speed,

enhancing application performance, and improving content indexability by search engines (SEO), making it suitable for educational platforms containing a large number of pages and courses.

3.2.1 Node.js

Node.js is used to run the application on the server and execute backend operations and process requests from users. It also provides a high-performance runtime environment based on an event-driven architecture, allowing the system to efficiently serve a large number of users.

3.2.2 MongoDB

The platform relies on MongoDB as a document-oriented (NoSQL) database, storing user, course, lesson, quiz, and student result data within flexible JSON documents, which facilitates system development and the addition of new features without requiring continuous modification of the database schema.

3.2.3 Gemini API

The system employs the Gemini API to provide artificial intelligence services, using it to generate quiz questions and, within the RAG framework, to generate accurate answers based on retrieved educational documents, thereby reducing the likelihood of producing incorrect or irrelevant information.

3.2.4 ChromaDB

The platform relies on ChromaDB as a vector database, storing embeddings of educational documents, and using them to perform semantic search operations and retrieve the passages most relevant to a student's query before sending them to the Gemini API.

3.2.5 Embedding Models

Embedding models are used to convert educational texts into numerical vectors representing their semantic meaning, enabling semantic search with higher accuracy than traditional keyword-based search. This component is the foundation on which RAG technology relies.

3.2.6 NextAuth.js

The system uses NextAuth.js to manage authentication processes and verify user identities, supporting secure login and management of sessions and the various permissions for students, instructors, and administrators.

3.2.7 Tailwind CSS

The project relies on Tailwind CSS for designing user interfaces, as it provides a flexible and rapid approach to building modern interfaces that are responsive across different screen sizes while maintaining design consistency and ease of maintenance.

3.3 Comparative Analysis of Existing Systems and Platforms

Table 10. Comparative Analysis of Learning Management System Features

Question / Feature	Moodle	Google Classroom	Chamilo LMS	Proposed System
Does the system support course management?	Yes	Yes	Yes	Yes
Does the system provide online quizzes?	Yes	Yes	Yes	Yes
Does the system provide role-based access (student/instructor/administrator)?	Yes	Yes	Yes	Yes
Is the system open-source and customizable?	Yes	No	Yes	Yes
Does the system automatically generate quizzes from uploaded documents using artificial intelligence?	No	No	No	Yes
Does the system integrate Retrieval-Augmented Generation (RAG) technology with course materials?	No	No	No	Yes
Does the system provide a dedicated analytics dashboard for instructors?	Yes	No	No	Yes

Question / Feature	Moodle	Google Classroom	Chamilo LMS	Proposed System
Does the system provide a dedicated analytics dashboard for administrators?	Yes	No	Yes	Yes
Does the system use Server-Side Rendering (SSR) to improve initial page load and search engine optimization?	No	No	No	Yes

3.4 Research Gap and Proposed Innovation

The reviewed studies show that most research has focused on addressing only a single aspect of intelligent learning systems; some studies focused on developing intelligent assistants based on Retrieval-Augmented Generation (RAG) technology to answer student inquiries, while other studies focused on using artificial intelligence models for automatic quiz generation, and yet others addressed learning analytics dashboards independently. Although these studies demonstrated the effectiveness of each technology individually, they did not present an integrated educational platform that combines these capabilities within a single learning management system.

It is also evident that the majority of systems and research efforts focused more heavily on improving the student experience by providing intelligent learning tools or virtual assistants, while attention to intelligent tools directed at instructors was comparatively limited, as most systems were restricted to course management functions or the display of traditional reports and statistics, without providing artificial-intelligence-based tools to help instructors prepare quizzes from educational content or support them in analyzing course data more effectively.

Building on these gaps, the current project provides an intelligent learning management system that combines several functions within a single platform, integrating an intelligent assistant based on RAG technology using the Gemini API to answer student inquiries based on course content, in addition to automatically generating quizzes from educational documents uploaded by instructors, while providing dedicated analytics dashboards for instructors and administrators to support monitoring of the educational process. The system also relies on Next.js with Server-Side Rendering (SSR) to improve application performance and enhance page indexability in search

engines, combining modern web development technologies with artificial intelligence technologies within a single scalable educational platform.

Thus, the project's main innovation lies in the integration of these functions within a single system that serves students, instructors, and administrators together, rather than offering each function separately, as is common in many previous studies and systems.

3.5 Recent and Future Trends

The field is undergoing rapid development as a result of integrating generative artificial intelligence technologies with educational platforms, contributing to a transition from systems limited to content management to systems capable of understanding content, interacting with it, and providing intelligent educational support. Among the most prominent recent trends in this field are:

- Generative AI: Now used in creating educational content, generating quizzes, answering student inquiries, and providing automated feedback, contributing to reducing the effort required from instructors and improving the learning experience.
- Retrieval-Augmented Generation (RAG): One of the most prominent recent trends in educational systems, combining semantic search with generative artificial intelligence models to produce answers based on reliable educational sources, thereby reducing hallucination and increasing the accuracy of answers.
- Large Language Models (LLMs): Large language models, such as those available through the Gemini API, are now used in developing educational assistants capable of understanding natural language, interpreting content, answering questions, and generating diverse educational materials.
- Intelligent Learning Analytics: Modern systems are moving toward developing data-driven analytics dashboards to support instructors and administrators in monitoring student performance, analyzing assessment results, and making more accurate educational decisions.
- Improving the performance of educational web applications: There is increasing reliance on modern frameworks that support Server-Side Rendering (SSR), such as Next.js, for the improvements they provide in page load speed, user experience, and content indexability

in search engines, which contributes to increasing access to educational content and improving platform performance.

Chapter 4. Requirements Analysis and System Modeling

4.1 Feasibility Study

4.1.1 Technical Feasibility

The technical feasibility of the project is high, as the system relies on modern, stable, and readily available technologies and tools. The project was implemented using open-source technologies and accessible cloud services.

4.1.2 Technical Requirements

- Programming Languages: JavaScript and TypeScript.
- Framework: Next.js and React.
- Runtime Environment: Node.js.
- Database: MongoDB.
- Vector Database: ChromaDB.
- Artificial Intelligence: Gemini API.
- Development Tools: Visual Studio Code, Git, GitHub.

4.1.3 Required Skills

- Modern web application development.
- Database design.
- Working with Application Programming Interfaces (APIs).
- Fundamentals of artificial intelligence and Retrieval-Augmented Generation (RAG) systems.
- Using Git for version control.

4.1.4 Hardware and Infrastructure

The system was developed using a personal computer with moderate specifications and an internet connection, without requiring servers or specialized equipment during the development phase, while the system can later be deployed on cloud hosting services.

4.1.5 Availability of Libraries and Tools

All libraries and frameworks used are freely available or open-source. The Gemini API also offers a free plan suitable for development and testing purposes, making the project feasible without significant technical obstacles.

Accordingly, the project is technically feasible, and there are no technical requirements that are unavailable or difficult to obtain.

4.1.6 Economic Feasibility

The project relied on free or low-cost tools, which significantly reduced development costs.

Table 11. Economic Feasibility: Development Tool Costs

Item	Approximate Cost
Visual Studio Code	Free
Node.js, React, and Next.js	Free
MongoDB Community	Free
ChromaDB	Free

Table 12. Economic Feasibility: Free and Low-Cost Tools

Item	Approximate Cost
GitHub and Git	Free
Gemini API (development phase)	Included in the free plan
Computer and internet connection	Already available

Total direct development cost: very low, as the project did not require the purchase of software licenses or additional equipment.

4.1.6.1 Expected Return

- Reducing the time required to create electronic tests.
- Improving the learning experience by providing an intelligent assistant based on course content.
- Increasing student engagement with educational content.
- Reducing the burden on faculty members in preparing assessments and answering repetitive questions.
- Potential future development into an educational platform usable across educational institutions.

Consequently, the expected benefits outweigh the development cost, making the project economically feasible.

4.1.7 Operational Feasibility

The system's interfaces were designed to be clear and suitable for both instructors and students, while reducing the number of steps required to accomplish core tasks.

4.1.7.1 Usability

- Simple and organized user interfaces.
- Ease of managing courses and lessons.
- Automatic test generation using artificial intelligence.
- Ability to ask questions to the intelligent assistant and receive answers based on course content.

4.1.7.2 Impact on Workflow

- Reducing the time required to prepare tests.
- Accelerating access to information within educational content.

- Supporting the educational process without a radical change in how the Learning Management System is used.
- Improving teaching efficiency and increasing instructor productivity.

Based on the results of implementing and testing the project, the system can operate within the current educational environment without requiring fundamental changes to the operational infrastructure, confirming the high operational feasibility of the project.

4.2 Identification and Analysis of Stakeholders

Table 13. Stakeholder Analysis: System Administrator Needs

Attribute	Description
Stakeholder	Needs and Expectations
Administrator	Managing users and permissions, monitoring the system, managing courses, and ensuring data security and system stability and performance.
Instructor	Creating and managing courses and lessons, uploading files, generating tests using artificial intelligence, and tracking student results.
Student	Accessing courses, taking tests, using the intelligent assistant (RAG) to obtain answers based on course content, and tracking academic progress.

4.3 Functional Requirements

1. FR1: The system shall support user registration.
2. FR2: The system shall support login and logout.
3. FR3: The system shall enforce access permissions according to the user's role.
4. FR4: The system administrator shall be able to manage user accounts.
5. FR5: The system administrator shall be able to activate and deactivate accounts.
6. FR6: The system administrator shall be able to manage academic courses.
7. FR7: The instructor shall be able to create a new course.
8. FR8: The instructor shall be able to edit course data.

9. FR9: The instructor shall be able to delete a course.
10. FR10: The instructor shall be able to create modules within a course.
11. FR11: The instructor shall be able to add lessons to modules.
12. FR12: The instructor shall be able to upload video files.
13. FR13: The instructor shall be able to upload DOCX files for lessons.
14. FR14: The system shall process and index file content.
15. FR15: The system shall store embeddings in the vector database.
16. FR16: The instructor shall be able to generate a test using artificial intelligence.
17. FR17: The instructor shall be able to specify the required number of questions.
18. FR18: The system shall support the generation of multiple-choice questions (MCQ).
19. FR19: The system shall support the generation of True/False questions.
20. FR20: The student shall be able to view the courses in which they are enrolled.
21. FR21: The student shall be able to view lessons and educational materials.
22. FR22: The student shall be able to take electronic tests.
23. FR23: The system shall automatically grade tests.
24. FR24: The system shall display the student's results after completing a test.
25. FR25: The student shall be able to ask questions to the intelligent assistant.
26. FR26: The intelligent assistant shall retrieve information from course content using RAG.
27. FR27: The intelligent assistant shall display answers based solely on course content.
28. FR28: The system shall maintain a log of important user operations.
29. FR29: The instructor shall be able to track student results and statistics.
30. FR30: The system administrator shall be able to monitor the system and manage its data.

4.4 Non-functional Requirements

Table 14. Non-functional Requirements and Measurement Criteria

Category	Non-functional Requirement (NFR)	Measurement Criterion
Performance	NFR1: The system page response time shall not exceed 2	≤ 2 seconds

Category	Non-functional Requirement (NFR)	Measurement Criterion
	seconds under normal operations.	
Performance	NFR2: The test generation time using the Gemini API shall not exceed 30 seconds for medium-sized files.	≤ 30 seconds
Performance	NFR3: The response time of the intelligent assistant (RAG) shall not exceed 10 seconds after a question is submitted.	≤ 10 seconds
Security	NFR4: Communication between the client and the server shall use the HTTPS protocol (TLS 1.2 or later).	100% of connections encrypted
Security	NFR5: Passwords shall be stored using the bcrypt algorithm and shall not be stored as plain text.	100% of passwords encrypted
Security	NFR6: The system shall enforce access permissions according to user roles (Admin, Instructor, Student).	100% prevention of unauthorized access
Usability	NFR7: The user shall be able to complete any primary task within a maximum of 3-5 steps.	≤ 5 steps
Usability	NFR8: The system interface shall be compatible with modern Chrome, Edge, and Firefox browsers.	Successful operation on all three browsers
Reliability	NFR9: The system shall achieve an availability rate of at least 99% during operation.	Availability ≥ 99%
Reliability	NFR10: User and test data shall be saved in the database without loss during normal operations.	100% data-saving success rate
Maintainability	NFR11: The system shall be based on a modular architecture to facilitate development and maintenance.	Ability to modify an independent module without affecting other modules

Category	Non-functional Requirement (NFR)	Measurement Criterion
Maintainability	NFR12: The code shall be documented, and Git shall be used for version control and change tracking.	All modifications documented in the Git repository
Scalability	NFR13: The system shall allow the addition of new courses and users without requiring modification of the core architecture.	Adding data without changing the architecture
Compatibility	NFR14: The system shall operate on Windows and Linux operating systems and through modern browsers.	Successful operation on target environments

4.5 Requirement Prioritization Using the MoSCoW Method

Table 15. Requirement Prioritization Using the MoSCoW Method

Priority	Requirements
Must	Login and authentication, user management, course and lesson management, uploading video and DOCX files, indexing lesson content in ChromaDB, generating tests using the Gemini API, support for multiple-choice questions (MCQ) and True/False questions, automatic test taking and grading, the RAG-based intelligent assistant, and permission management according to user role (Admin, Instructor, Student).
Should	Displaying student results and statistics, activity logging (Logs), profile management, user interface improvement, and improving search and response performance.
Could	Exporting results to PDF or Excel, sending in-system notifications, an advanced analytics dashboard, and customizing course settings.
Won't	A smartphone application, support for adaptive learning, support for offline operation, support for multiple languages, and integration with external Learning Management Systems such as Moodle or Blackboard.

4.6 Requirements Modeling Using UML

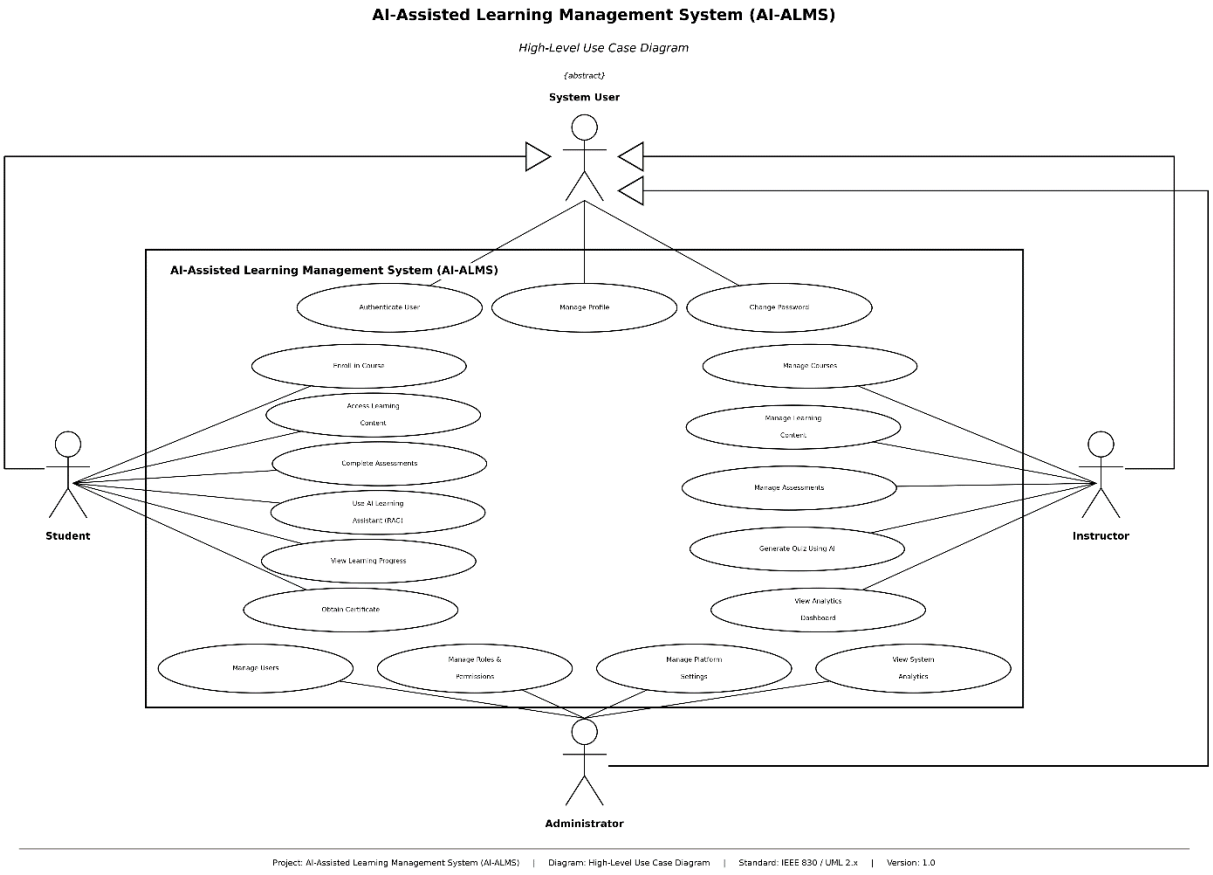


Figure 2. Use Case Diagram of the Proposed System

4.6.1 UC1 - Manage Courses & Lessons

Table 16. Use Case UC1

Attribute	Specification
Use Case ID	**UC1**
Use Case Name	Manage Courses & Lessons
Actor(s)	Instructor
Preconditions	1. Instructor is authenticated and has the Instructor role. 2. iinstructor has access to the dashboard.
Postconditions	Course with its modules, lessons, and uploaded assets (video + DOCX) is persisted in the database and optionally published to the public catalog.
Primary Path	1. Instructor opens the course management dashboard. 2. Instructor clicks "Add Course" and enters course metadata (title, description, category, thumbnail image). 3. Instructor creates one or more modules within the course. 4. Instructor creates lessons inside each module, optionally reordering them via drag-and-drop. 5. Instructor uploads a video file and a DOCX lecture document for each lesson. 6. System stores assets, records metadata in MongoDB, and confirms success. 7. Instructor publishes the course to the public catalog.
Alternative Paths	A1 – Edit existing course: At Step 1, instructor selects an existing course → modifies fields → saves changes. A2 – Reorder modules/lessons: At Step 4, instructor uses drag-and-drop to reorder modules or lessons within a module. A3 – Upload failure: At Step 5, if the upload fails, the system displays an error toast notification; instructor retries or selects a different file. A4 – Unpublish course: At Step 7, instructor unpublishes a live course; existing enrolled students retain access, but new enrollments are blocked.

Attribute	Specification
Exceptions	E1. Unauthorized access: system redirects to login if session is expired. E2. File format not supported: system rejects the upload with a validation error.

4.6.2 UC2 - Generate Quiz from a Document Using Artificial Intelligence

Table 17. Use Case UC2

Attribute	**Specification**
Use Case ID	UC2
Use Case Name	Generate Quiz from a Document Using Artificial Intelligence
Primary Actor	Instructor
Preconditions	1. Instructor is authenticated. 2. Instructor has course management privileges. 3. A course exists. 4. • A DOCX document is available, either linked to a lesson or uploaded by the instructor.
Postconditions	1. A quiz draft is successfully generated. 2. The instructor can review, edit, regenerate, approve, or delete generated questions. 3. The quiz is saved as an unpublished quiz within the course.
Main Flow	1. Instructor opens the course Quizzes page. 2. Clicks "Generate from Document". 3. System displays quiz generation page. 4. Instructor selects a lesson document or uploads a DOCX file. 5. Instructor specifies generation settings (total questions, MCQ, True/False, difficulty distribution). 6. Clicks "Generate Draft". 7. System generates the quiz using AI. 8. System displays generated questions with type, difficulty, options, correct answer, and explanation. 9. Instructor reviews and edits/regenerates/approves/deletes questions.10. Clicks "Save Quiz as Draft". 11. Enters quiz title, optional description, and passing score. 12. Confirms save. 13. System saves the quiz as Unpublished.

Attribute	**Specification**
Alternative Flows	A1 – Use Existing Lesson Document: If a lesson with an associated DOCX is selected, the system uses that document. A2 – Upload New Document: Otherwise, the instructor uploads a new DOCX document. A3 – Regenerate Questions: The instructor may regenerate any generated question. A4 – Delete Questions: The instructor may delete unnecessary questions before saving.
Exceptions	E1 – Missing Document: If no DOCX document is selected or uploaded, the system prevents generation and requests a document. E2 – AI Generation Failure: If AI-based generation fails, the system displays an error message and allows the instructor to retry.

4.6.3 UC3 - Enroll & Learn

Table 18. Use Case UC3 Attributes

Attribute	Specification
Use Case ID	**UC3**
Use Case Name	Enroll & Learn
Actor(s)	Student
Preconditions	1. Student is authenticated. 2. Course is published and visible in the catalog. 3. Student is not already enrolled in the selected course.
Postconditions	Student is enrolled in the course, can access all lessons, and watch progress is tracked toward completion.
Primary Path	1. Student browses the public course catalog. 2. Selects a course to view details. 3. Student Clicks "Enroll" and proceeds to mock checkout. 4. Student Completes payment; system confirms enrollment. 5. Student opens a lesson; system streams video 6. Student watches; system records progress.

Attribute	Specification
Alternative Paths	A1 (Already Enrolled): Button displays "Continue Learning" and jumps to the last-watched lesson. A2 (Payment Fails): System displays an error; student can retry. A3 (Text-to-Video Jump): Student clicks a synced text paragraph; video seeks to the exact timestamp. A4 (Certificate): Student requests a certificate at 100% progress; system generates a rate-limited PDF.
Exceptions	E1: Course is unpublished mid-enrollment; system displays "Course not available." E2: Video file is corrupted; system displays an error state.

4.6.4 UC4 - Ask RAG Tutor

Table 19. Use Case UC4 Attributes

Attribute	Specification
Use Case ID	UC4
Use Case Name	Ask RAG Tutor
Actor(s)	Student
Preconditions	1. Student is authenticated and enrolled. 2. Lesson content is indexed in ChromaDB.
Postconditions	Student receives a grounded answer with timestamp links; interaction is logged; rate-limit is incremented.

Attribute	Specification
Primary Path	1. Student clicks "Ask Tutor"; video pauses. 2. Student types a question. 3. System verifies authorization. 4. System checks rate limits, generates an embedding, and retrieves the top chunks from ChromaDB. 5. System generates a response strictly grounded in the retrieved context. 6. System displays the answer with timestamp links and checks if a recite-back is required.
Alternative Paths	A1 (No Content): System replies that the topic is not covered and suggests alternatives. A2 (Limit Reached): System issues a rate-limit warning after 10 questions in 24 hours. A3 (ChromaDB Offline): System returns a degraded state indicating content is not indexed.
Exceptions	E1: Unenrolled student receives a 403 Forbidden response.

4.6.5 UC5 - Platform Analytics and Dashboard Customization

Table 20. Use Case UC5 Attributes

Attribute	**Specification**
Use Case ID	UC2
Use Case Name	Platform Analytics & Dashboard Customization
Actor(s)	Administrator (Admin)
Preconditions	1. The Administrator is authenticated and possesses valid Admin role privileges. 2. The Administrator navigates to the "Analytics" section via the admin panel sidebar (/admin/analytics).

Attribute	Specification
Postconditions	1. Updated metrics and visual data charts are rendered according to the selected date range or active tab. 2. User dashboard customization preferences (section visibility and tab ordering) are persisted in the system. 3. Analytics data reports are successfully generated and downloaded as CSV files upon request.
Primary Path	1. The Administrator accesses the Platform Analytics dashboard (/admin/analytics). 2. The system loads the default "Overview" tab, displaying high-level key metrics (Total Courses, Total Users, Total Revenue, and Active Users). 3. The Administrator navigates across different analytics tabs (Users, Courses, Revenue, Performance, Projections). 4. The Administrator selects a predefined date range filter (Last 7 days, Last 30 days, Last 90 days) or sets a Custom range. 5. The system dynamically updates and re-renders all metrics and charts based on the selected criteria. 6. The Administrator clicks the "Customize dashboard" button to open the dashboard customization modal. 7. The Administrator modifies section visibility (toggle on/off), reorders tabs, or changes the default date range filter, then clicks "Save". 8. The system saves the configuration settings and updates the dashboard layout accordingly.
Alternative Paths	A1 – Export Data (Export CSV): At Step 4, the Administrator clicks "Export CSV" -> The system compiles the current dataset and downloads a CSV report to the user's device. A2 – Manual Data Refresh (Refresh Data): The Administrator clicks the "Refresh Data" button -> The system immediately fetches and displays the latest analytics metrics from the database. A3 – Modify Data Aggregation (Daily / Weekly / Monthly): When viewing specific tabs (e.g., Users or Courses), the Administrator switches the aggregation granularity between Daily, Weekly, and Monthly views.

Attribute	Specification
Exceptions	E1. Session Expiration: If the Administrator's session expires or authentication fails, the system redirects the user to the Login page. E2. No Data Available: If no data exists for the selected date range or filters, the system displays a "No data for this date range" placeholder within chart containers while maintaining UI stability.

4.6.6 UC6 - Instructor Course Analytics and Customization

Table 21. Use Case UC6 Attributes

Attribute	Specification
Use Case ID	UC6
Use Case Name	Instructor Course Analytics & Customization
Actor(s)	Instructor
Preconditions	1. The Instructor is authenticated and possesses valid Instructor role privileges. 2. The Instructor navigates to the Course Analytics page via the instructor dashboard sidebar (/dashboard/analytics).
Postconditions	1. Course analytics data, student progress metrics, activity trends, and earnings breakdown are rendered according to the applied filters. 2. User dashboard layout preferences (section visibility and default date range) are persisted in the system. 3. Student progress rosters or earnings data reports are successfully exported and downloaded as CSV files upon request.

Attribute	Specification
Primary Path	1. The Instructor accesses the Course Analytics dashboard (/dashboard/analytics). 2. The system loads the default "Overview" tab, displaying key summary metrics (Total Courses, Total Enrollments, Active Students, and Total Revenue). 3. The Instructor navigates across available analytics tabs (Students, Activity, Earnings). 4. The Instructor selects a specific date range filter (Last 7 days, Last 30 days, Last 90 days, or Custom range). 5. The system dynamically updates and re-renders all metrics and data visualizations based on the selected filter criteria. 6. The Instructor clicks the "Customize dashboard" button to launch the layout configuration modal. 7. The Instructor toggles section visibility (Overview, Students, Activity, Earnings), reorders dashboard sections, sets the default date range filter, and clicks "Save". 8. The system saves the configuration settings and updates the dashboard view accordingly.
Alternative Paths	A1 – Filter Student Progress Data: Under the "Students" tab, the Instructor selects a specific course from the dropdown menu and filters students by completion status (All, Not started, In progress, Near completion, Completed). A2 – Track Student Activity & Engagement: Under the "Activity" tab, the Instructor reviews average session duration, inactive student metrics, and login activity distribution charts organized by days of the week and hours of the day. A3 – View and Analyze Earnings: Under the "Earnings" tab, the Instructor inspects average earnings per student, earnings trend lines, and course-wise revenue distribution, with options to adjust temporal granularity (Daily, Weekly, Monthly). A4 – Export Data (Export CSV): The Instructor clicks "Export CSV" within the Students or Earnings tab -> The system compiles the active dataset and downloads a CSV report to the user's local device.

Attribute	Specification
Exceptions	E1. No Data Available for Selected Criteria: If no analytics or earnings data exists for the selected filter or date range, the system displays zeroed metrics or explanatory placeholders without interrupting UI stability. E2. Session Expiration: If the Instructor's authentication session expires while interacting with the dashboard, the system redirects the user to the Login page.

To delineate the precise behavioral logic of the AI-Assisted Learning Management System (AI-ALMS), activity diagrams have been constructed for the six primary use cases. Where necessary, swim lanes are utilized to separate user actions from automated system processes.

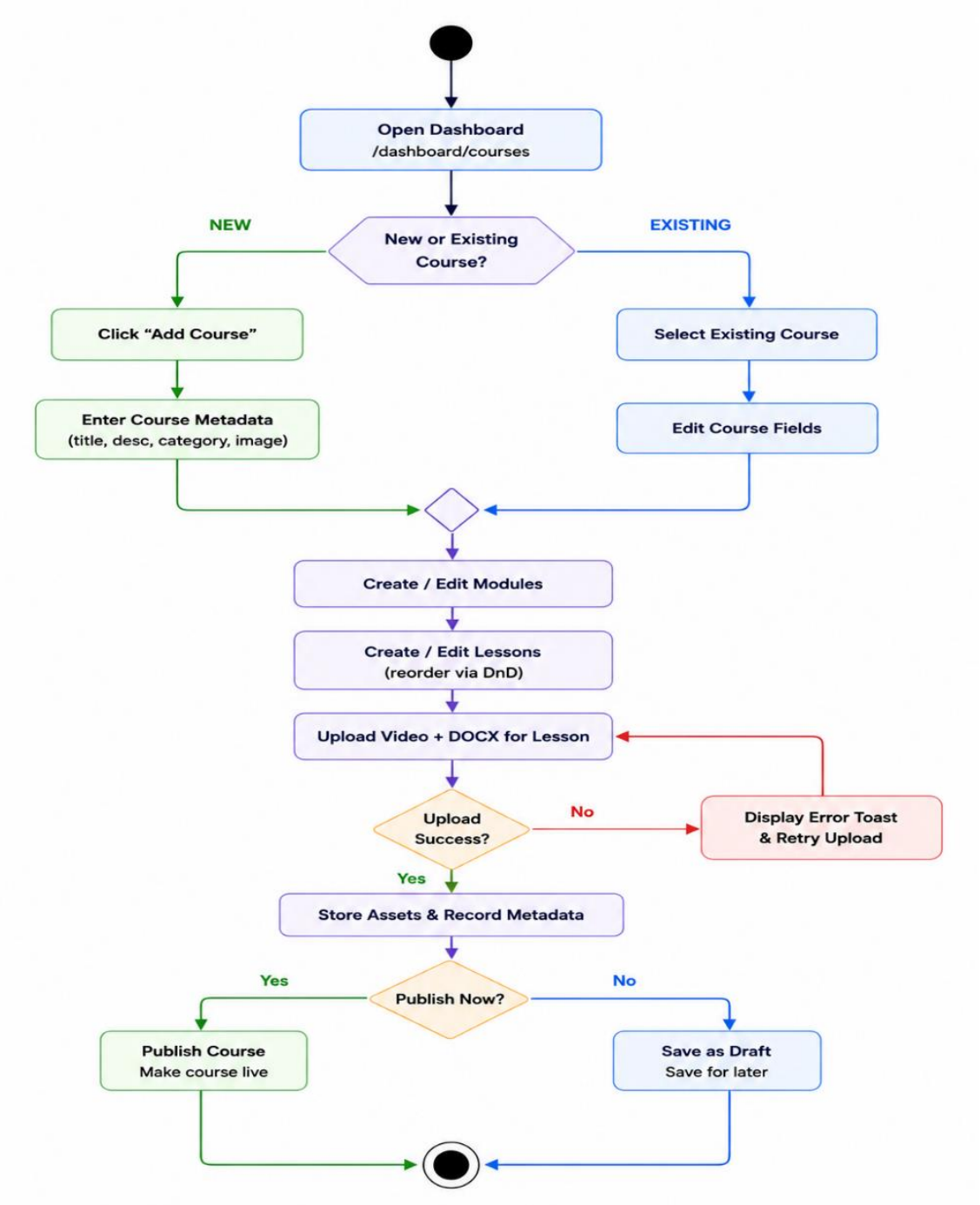


Figure 3. Activity Diagram: Manage Courses & Lessons

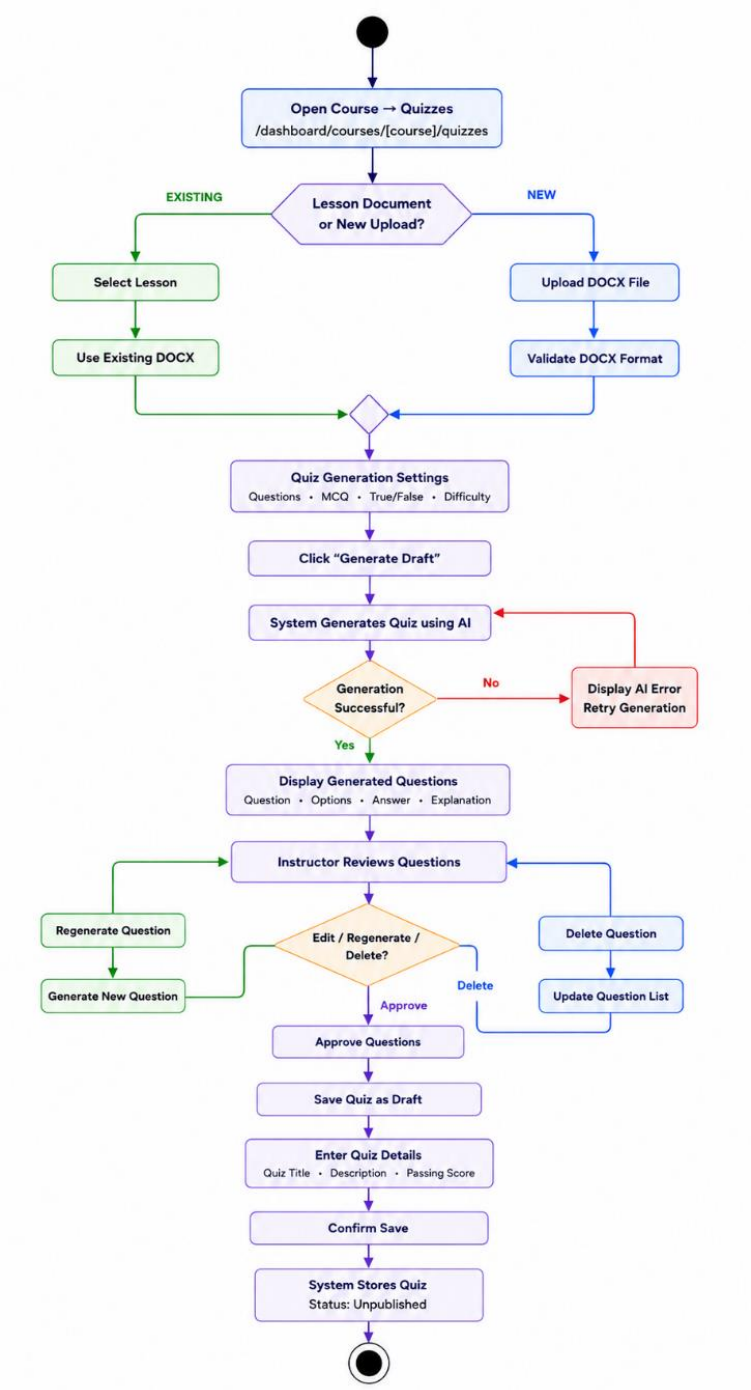


Figure 4. Activity Diagram: Generate Quiz from a Document Using Artificial Intelligence

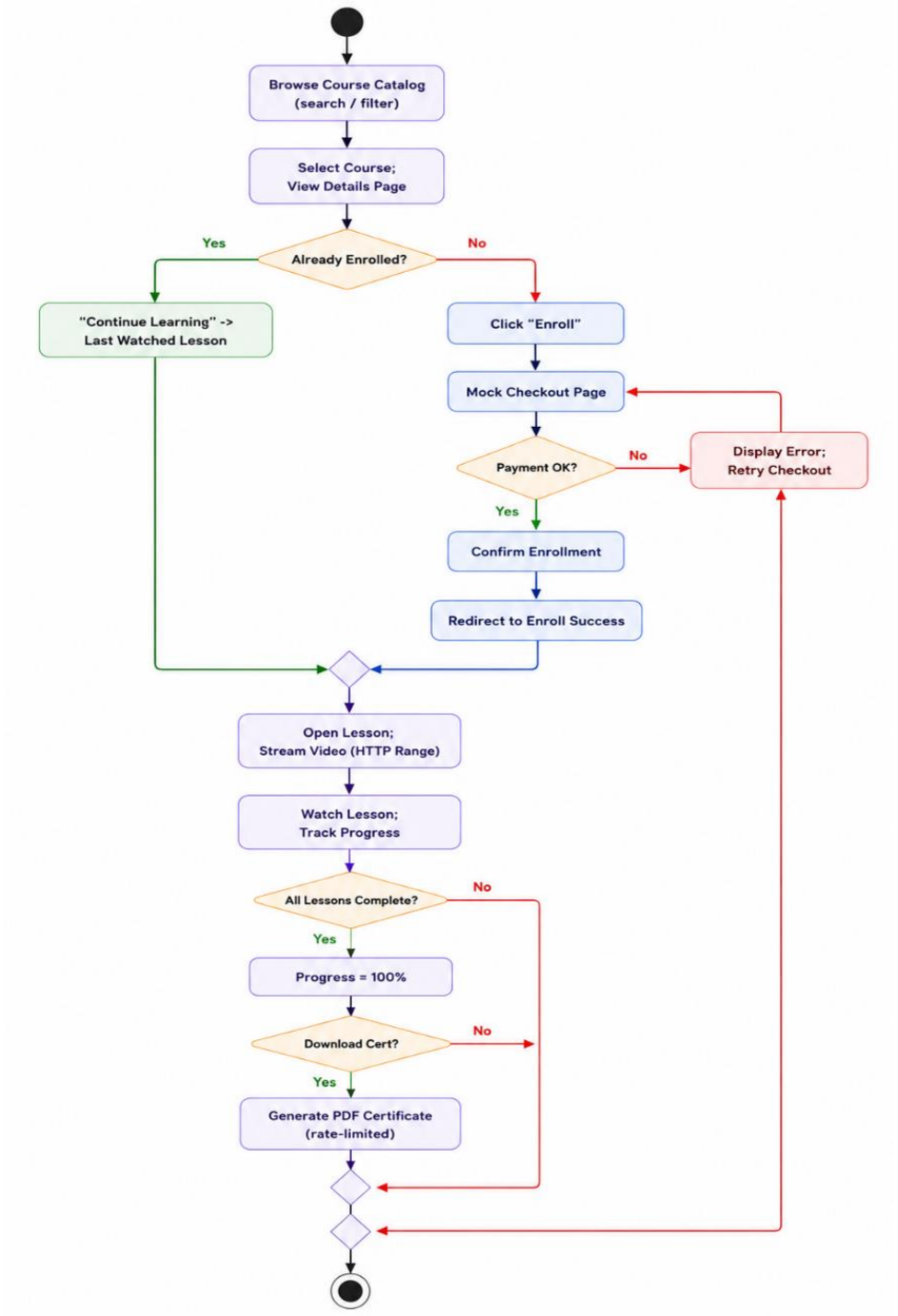


Figure 5. Activity Diagram: Enroll & Learn

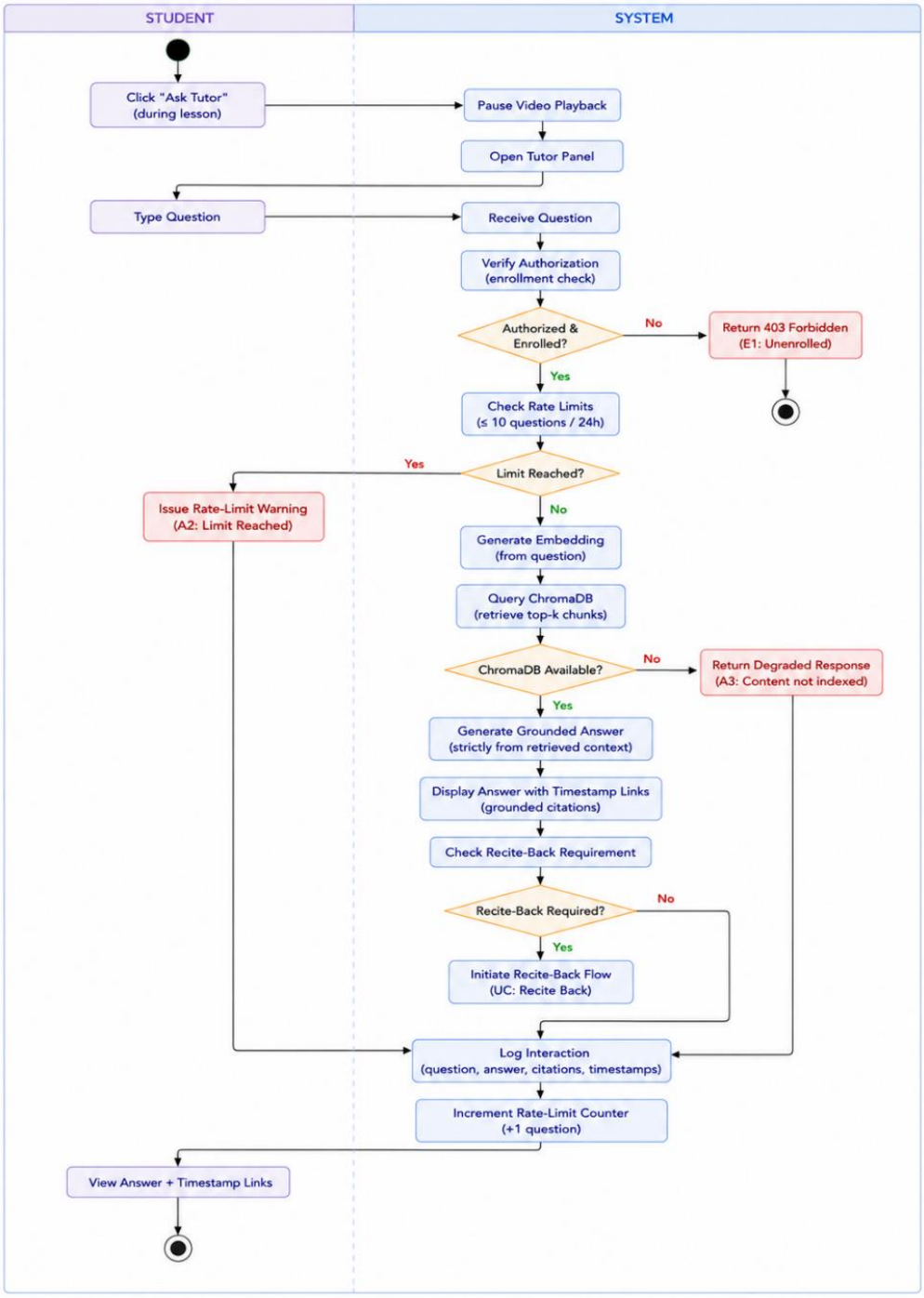


Figure 6. Activity Diagram: Ask RAG Tutor

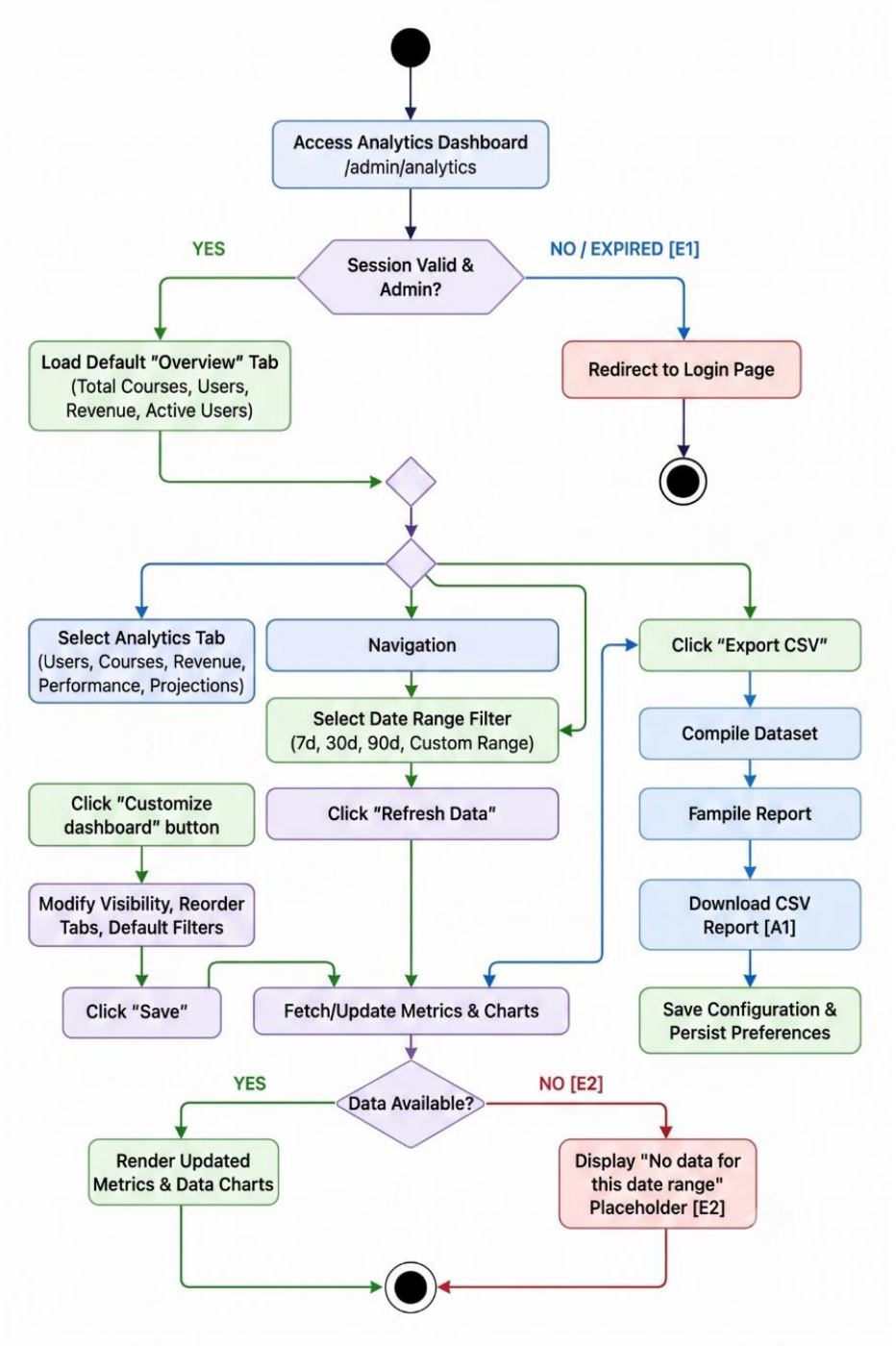


Figure 7. Activity Diagram: Platform Analytics & Dashboard Customization

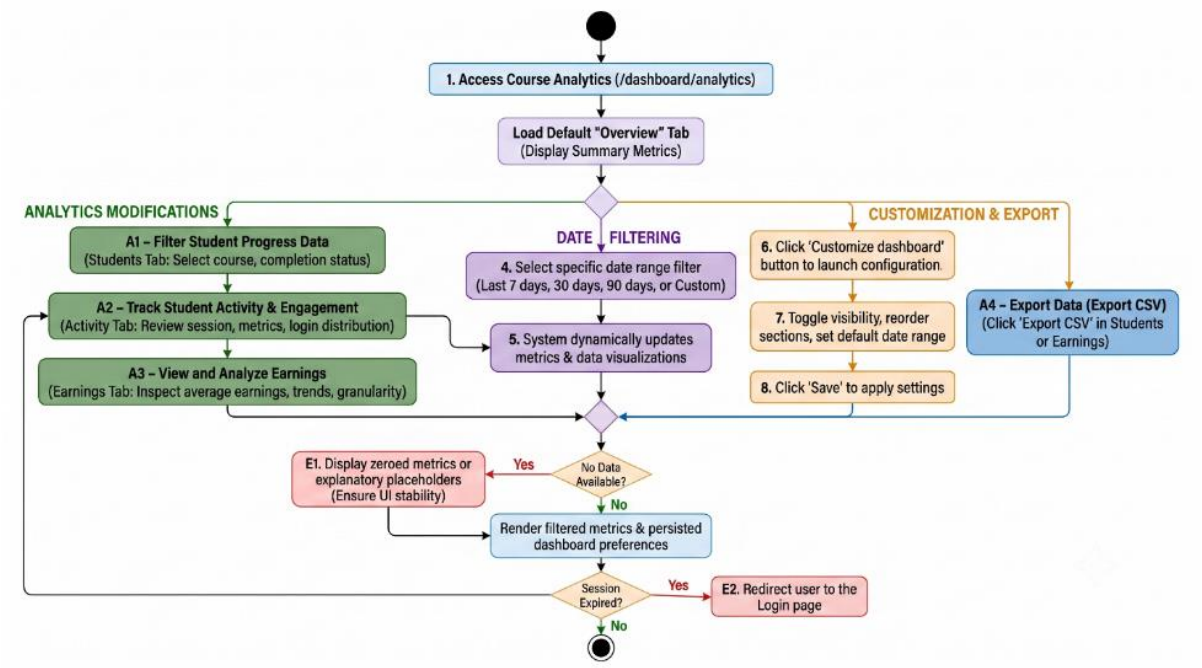


Figure 8. Activity Diagram: Instructor Course Analytics & Customization

4.6.7 System Sequence Diagrams

To capture the precise temporal order of interactions within the AI-ALMS system, sequence diagrams have been constructed for the six primary use cases. These models define the dynamic flow of messages between system actors (users) and the underlying architectural components (API routes, server actions, services, and databases).

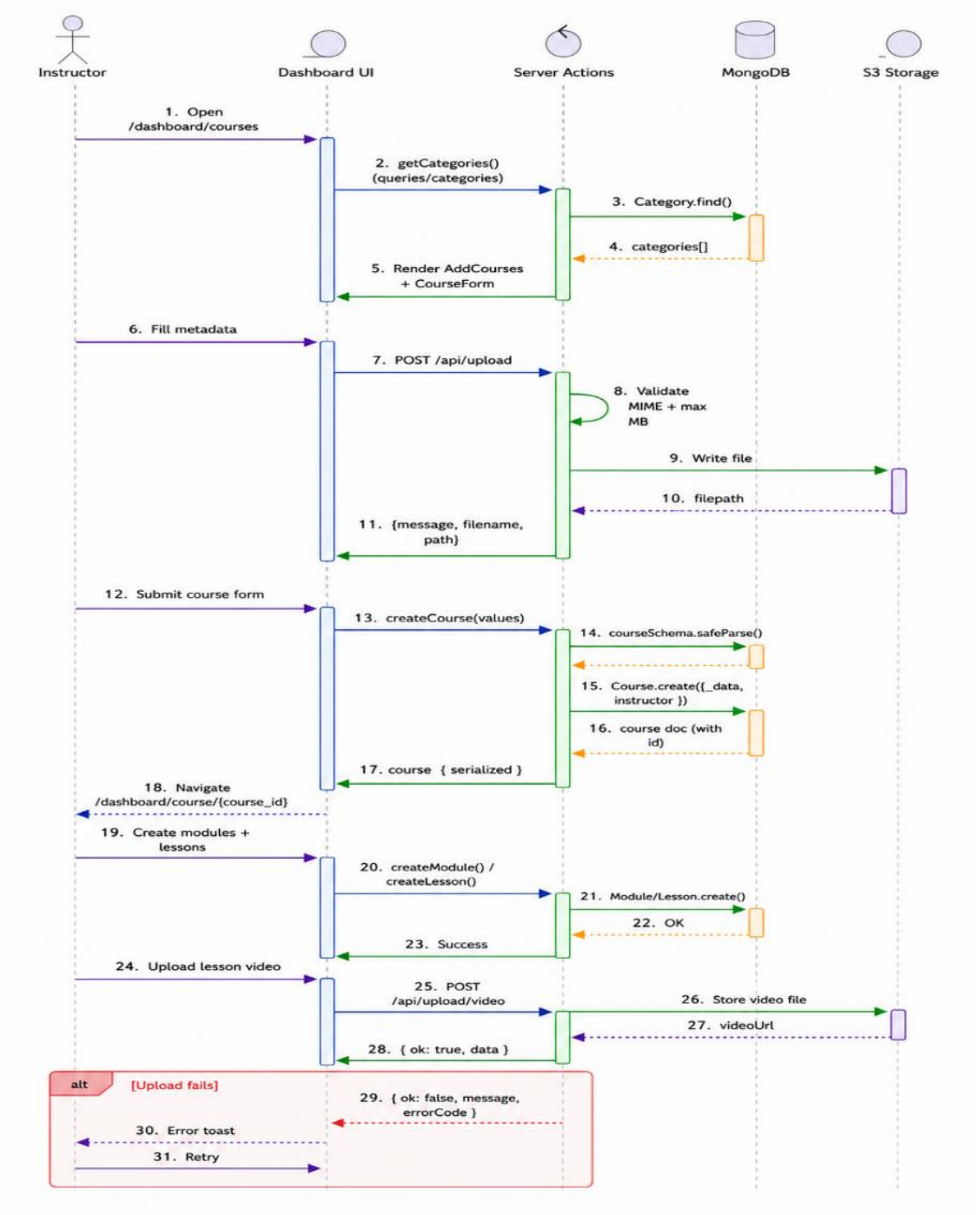


Figure 9. Sequence Diagram: Manage Courses & Lessons

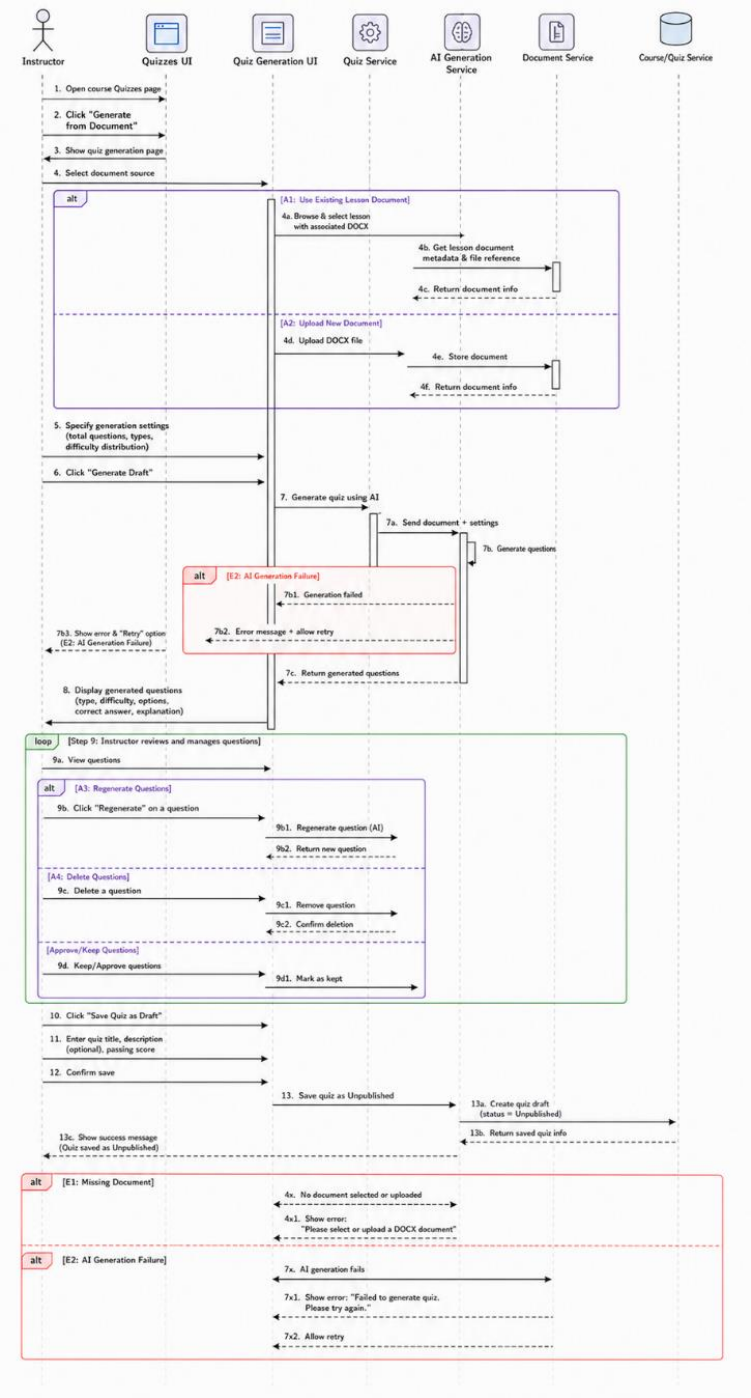


Figure 10. Sequence Diagram: Generate Quiz from a Document Using Artificial Intelligence

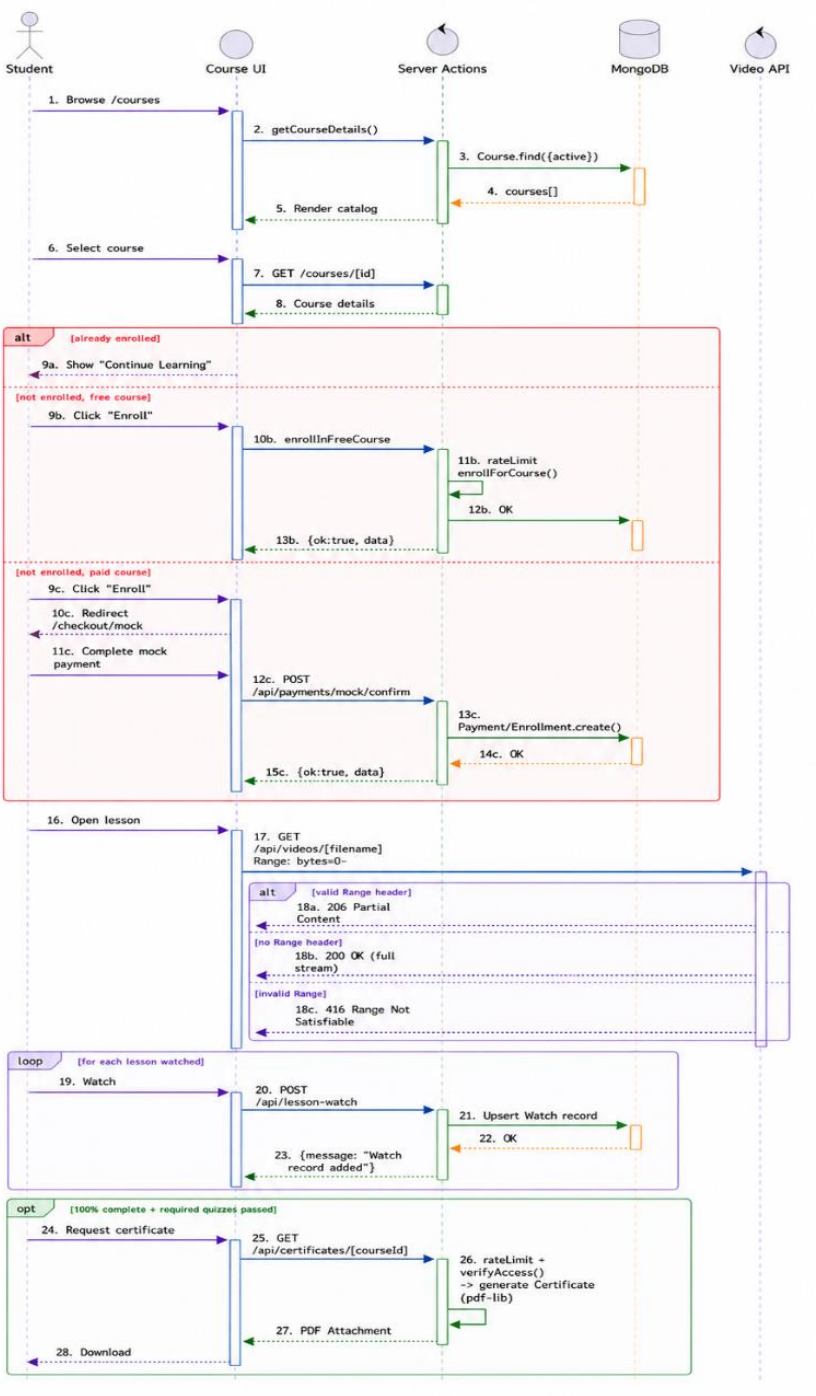


Figure 11. Sequence Diagram: Enroll & Learn

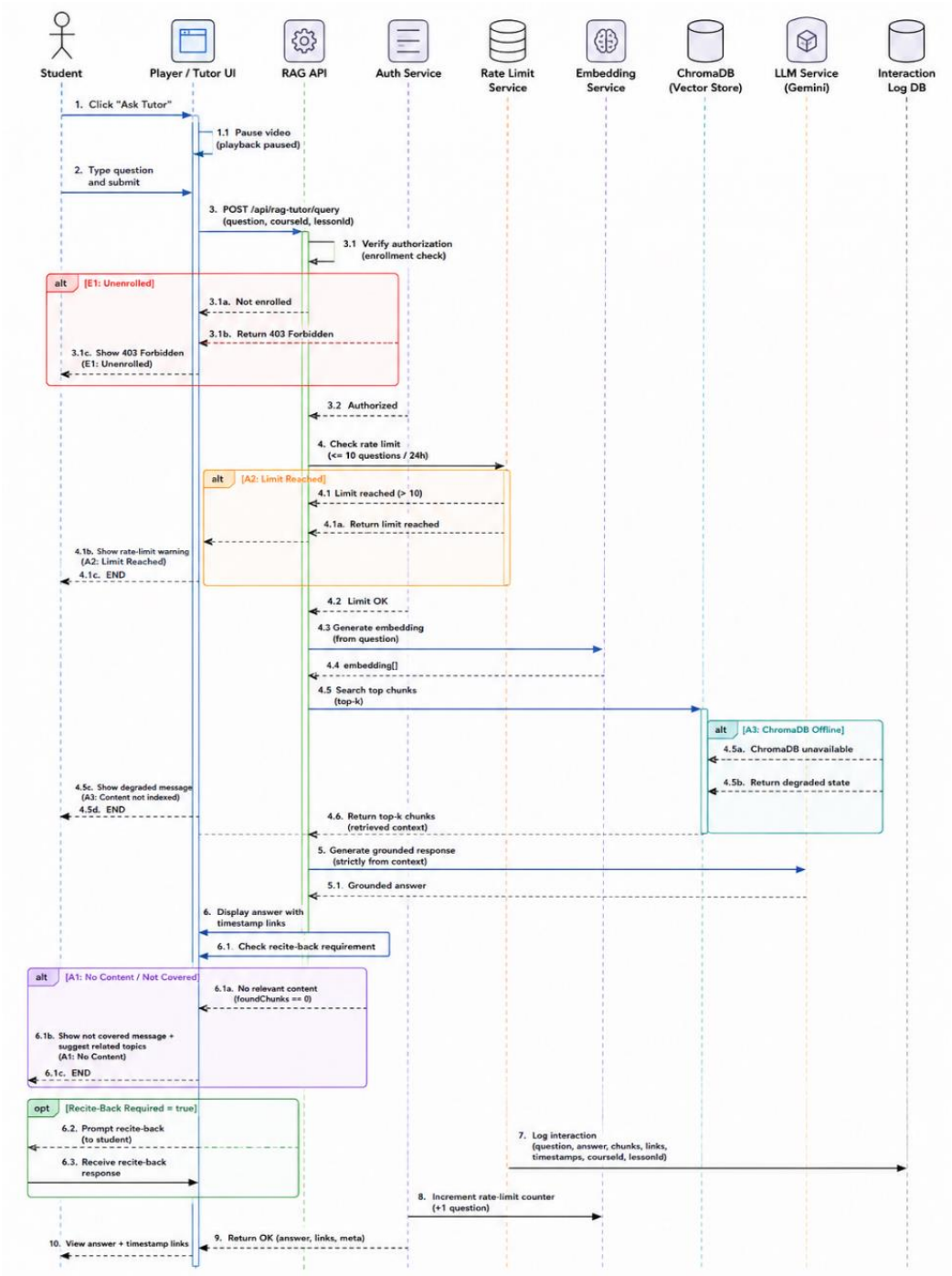


Figure 12. Sequence Diagram: Ask RAG Tutor

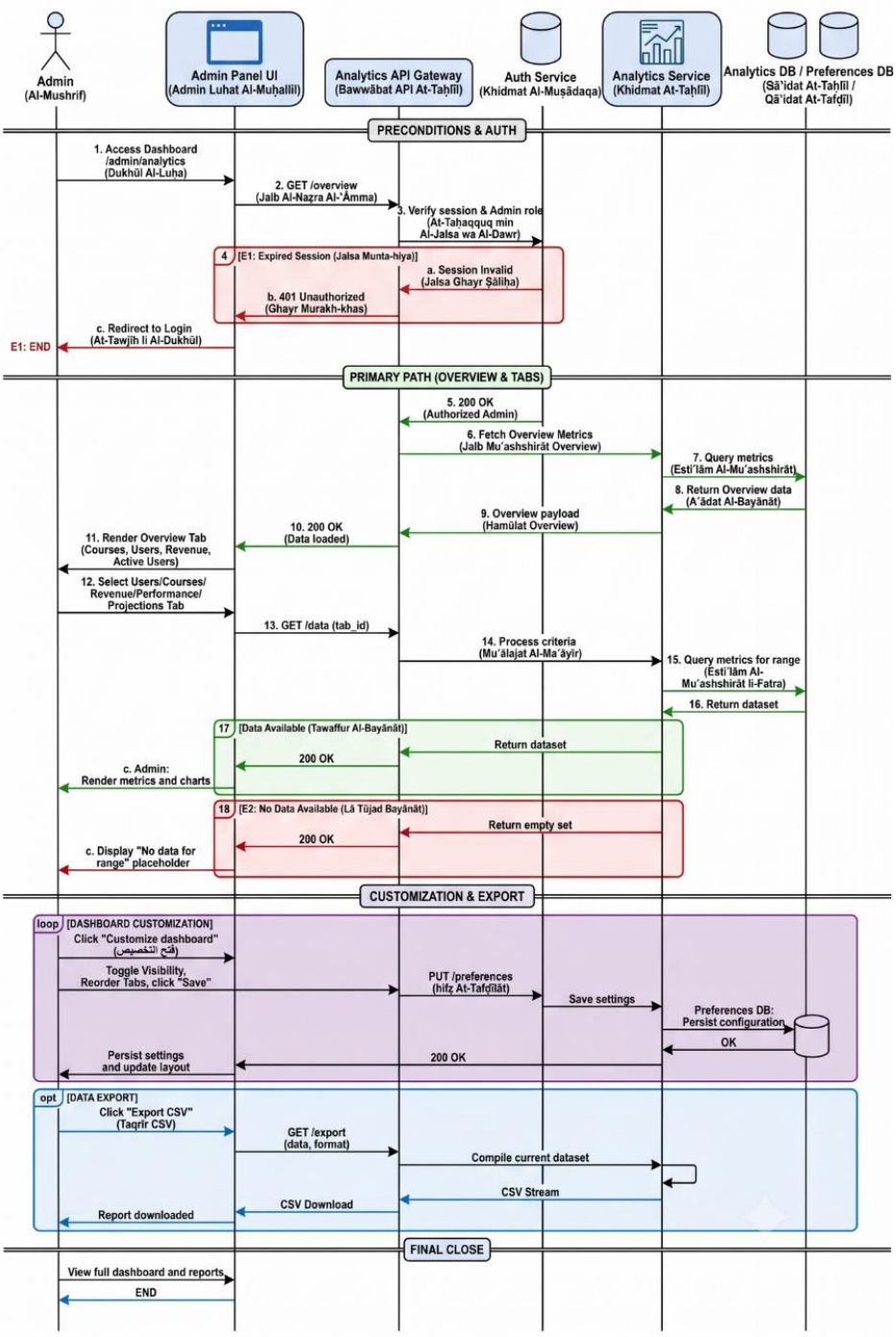


Figure 13. Sequence Diagram: Platform Analytics & Dashboard Customization

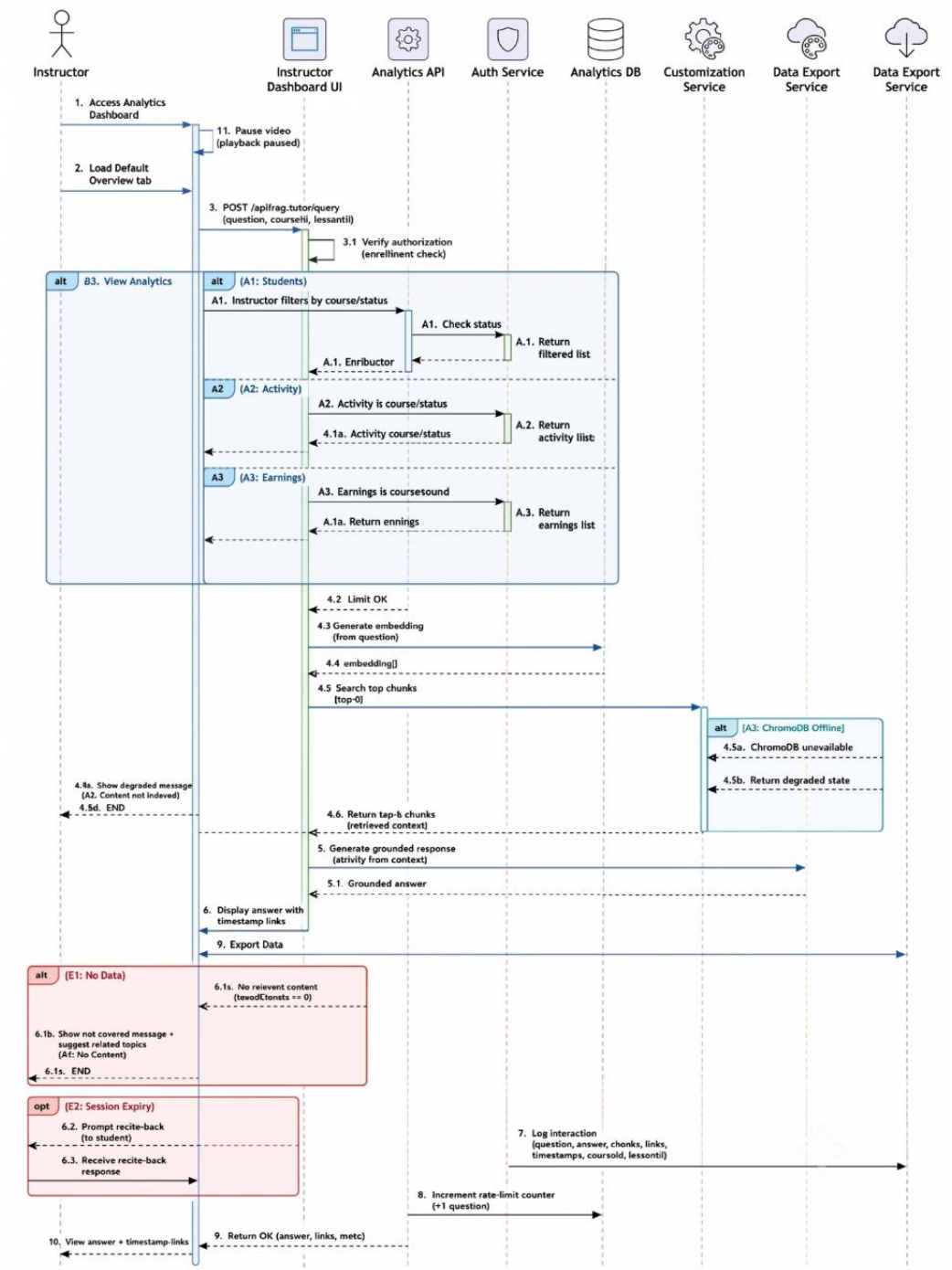


Figure 14. Sequence Diagram: Instructor Course Analytics & Customization

4.7 System Modeling According to the Incremental Development Methodology

Table 22. Incremental Development Plan

Increment	ID	Task	Timeframe	Development Phase	Details and Activities	Milestone
Increment 1 (Assessment Automation)	T1	Word Text Extraction & Parser Development	Jun 20 - Jun 23	Analysis and Implementation	Word Text Extraction & Parser Development	Automated Quiz Generation & Validation
Increment 1 (Assessment Automation)	T2	Gemini API Prompt Engineering & Configuration	Jun 24 - Jun 27	Design and Implementation	Gemini API Prompt Engineering & JSON Mode Configuration	Automated Quiz Generation & Validation
Increment 1 (Assessment Automation)	T3	MongoDB Schema Update & Data Integration	Jun 28 - Jul 01	Design and Implementation	MongoDB Quiz Schema Update & Data Integration	Automated Quiz Generation & Validation
Increment 1 (Assessment Automation)	T4	Instructor Review Interface & Unit Testing	Jul 02 - Jul 05	Interfaces and Testing	Instructor Review UI Development & Unit Testing	Automated Quiz Generation & Validation
Increment 2 (Student RAG Agent)	T1	Video Lecture Transcription & Chunking	Jul 06 - Jul 09	Analysis and Implementation	Video Lecture Transcription & Chunking Pipeline	Intelligent Student Conversational Assistant
Increment 2 (Student RAG Agent)	T2	Embeddings Generation & Vector Search	Jul 10 - Jul 13	Design and Implementation	Embeddings Generation & MongoDB Vector Search	Intelligent Student Conversational Assistant
Increment 2 (Student RAG Agent)	T3	Context Constraints & Injection Protection	Jul 14 - Jul 17	Design and Security	Context Guardrails & Prompt Injection Prevention	Intelligent Student Conversational Assistant
Increment 2 (Student	T4	Conversation Interface &	Jul 18 - Jul 21	Interfaces and Testing	Conversational Chat UI &	Intelligent Student

Increment	ID	Task	Timeframe	Development Phase	Details and Activities	Milestone
RAG Agent)		Retrieval Testing			Retrieval Testing	Conversational Assistant
Increment 3 (Analytics Dashboards & Integration)	T1	Admin Dashboard	Jul 22 - Jul 25	Design and Implementation	User statistics, course statistics, and system overview	Admin & Instructor Analytics Dashboards & Final Handover
Increment 3 (Analytics Dashboards & Integration)	T2	Instructor Dashboard	Jul 26 - Jul 29	Design and Implementation	Student progress analytics, quiz performance analytics, and course insights	Admin & Instructor Analytics Dashboards & Final Handover
Increment 3 (Analytics Dashboards & Integration)	T3	Dashboard APIs	Jul 30 - Aug 02	Implementation and Optimization	MongoDB aggregation pipelines, data integration, and performance optimization	Admin & Instructor Analytics Dashboards & Final Handover
Increment 3 (Analytics Dashboards & Integration)	T4	Comprehensive Testing & Final Handover	Aug 03 - Aug 07	Testing and Handover	Bug fixes, documentation, and final handover	Admin & Instructor Analytics Dashboards & Final Handover

4.8 Requirements Analysis and Documentation

4.8.1 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) establishes bidirectional traceability between the functional requirements, the corresponding UML design artifacts, and the verification test cases. This ensures that every functional requirement is reflected in the system design and validated during testing:

FR ID	Functional Requirement	Design Artifact	Test Case	Status
FR1	Secure authentication (Login / Logout / Register)	UC1, SD1	TC-1.1	Verified
FR2	Role-Based Access Control	UC1, UC5, UC6	TC-1.2	Verified
FR3	Manage user accounts	UC6, SD6	TC-6.1	Verified
FR4	Create, update and delete courses	UC1, AD1, SD1	TC-1.3	Verified
FR5	Create and manage modules and lessons	UC1, AD1, SD1	TC-1.4	Verified
FR6	Upload lesson videos	UC1, SD1	TC-1.5	Verified
FR7	Upload DOCX lesson documents	UC1, SD1	TC-1.6	Verified
FR8	Extract text from uploaded DOCX files	UC2, AD2, SD2	TC-2.1	Verified
FR9	Generate semantic embeddings	UC2, SD2	TC-2.2	Verified
FR10	Store embeddings in ChromaDB	UC2, SD2	TC-2.3	Verified
FR11	Generate quizzes using Gemini API	UC2, AD2, SD2	TC-2.4	Verified
FR12	Generate Multiple Choice Questions	UC2	TC-2.5	Verified
FR13	Generate True/False Questions	UC2	TC-2.6	Verified
FR14	Review generated questions	UC2	TC-2.7	Verified
FR15	Allow students to access enrolled courses	UC3, AD3, SD3	TC-3.1	Verified
FR16	Restrict lesson access to enrolled students	UC3	TC-3.2	Verified
FR17	Stream lesson videos securely	UC3	TC-3.3	Verified
FR18	Allow students to attempt quizzes	UC3	TC-4.1	Verified
FR19	Automatically grade quiz submissions	UC3	TC-4.2	Verified
FR20	Display quiz scores and feedback	UC3	TC-4.3	Verified
FR21	Answer student questions using RAG	UC4, AD4, SD4	TC-5.1	Verified
FR22	Retrieve relevant lesson chunks	UC4	TC-5.2	Verified
FR23	Generate grounded answers using Gemini	UC4	TC-5.3	Verified
FR24	Display source references	UC4	TC-5.4	Verified
FR25	Log important user activities	UC5	TC-6.2	Verified
FR26	Allow administrators to manage courses and users	UC5	TC-6.3	Verified
FR27	Display instructor analytics	UC6, AD6, SD6	TC-6.4	Verified
FR28	Display administrator analytics	UC5, AD5, SD5	TC-6.5	Verified
FR29	Maintain quiz and course data in MongoDB	UC1–UC6	TC-6.6	Verified
FR30	Support secure session management	UC1–UC6	TC-6.7	Verified

Table 23. Requirements Traceability Matrix

Initial Test Cases:

Test ID	Requirement	Test Description	Expected Result	Reference
TC-1.1	FR1	Verify secure user authentication using valid credentials.	User is authenticated successfully and redirected to the dashboard.	Chapter 7
TC-1.2	FR2	Verify role-based access permissions.	Users can access only resources permitted by their assigned role.	Chapter 7
TC-1.3	FR4	Verify course creation.	Course is created successfully and stored in the database.	Chapter 7
TC-1.4	FR5	Verify module and lesson creation.	Modules and lessons are created successfully.	Chapter 7
TC-1.5	FR6	Verify lesson video upload.	Video is uploaded and linked to the lesson.	Chapter 7
TC-1.6	FR7	Verify DOCX upload.	Document is uploaded successfully.	Chapter 7
TC-2.1	FR8	Verify DOCX text extraction.	Text is extracted successfully.	Chapter 7
TC-2.2	FR9	Verify embedding generation.	Embeddings are generated successfully.	Chapter 7
TC-2.3	FR10	Verify ChromaDB storage.	Embeddings are stored and retrievable.	Chapter 7
TC-2.4	FR11	Verify AI quiz generation.	Gemini generates quiz questions successfully.	Chapter 7
TC-2.5	FR12	Verify MCQ generation.	MCQ questions are generated correctly.	Chapter 7
TC-2.6	FR13	Verify True/False generation.	True/False questions are generated correctly.	Chapter 7
TC-2.7	FR14	Verify question review.	Instructor reviews and publishes questions successfully.	Chapter 7
TC-3.1	FR15	Verify enrolled course access.	Student accesses only enrolled courses.	Chapter 7
TC-3.2	FR16	Verify lesson authorization.	Unauthorized users are denied access.	Chapter 7
TC-3.3	FR17	Verify secure video streaming.	Video streams successfully.	Chapter 7
TC-4.1	FR18	Verify quiz submission.	Quiz is submitted successfully.	Chapter 7

TC-4.2	FR19	Verify automatic grading.	Quiz score is calculated correctly.	Chapter 7
TC-4.3	FR20	Verify quiz result display.	Student can view score and feedback.	Chapter 7
TC-5.1	FR21	Verify RAG tutor query.	Context-grounded answer is generated.	Chapter 7
TC-5.2	FR22	Verify semantic retrieval.	Relevant document chunks are retrieved.	Chapter 7
TC-5.3	FR23	Verify AI response generation.	Gemini generates a grounded response.	Chapter 7
TC-5.4	FR24	Verify source reference display.	Generated response includes source references.	Chapter 7
TC-6.1	FR3	Verify user management.	Administrator manages user accounts successfully.	Chapter 7
TC-6.2	FR25	Verify activity logging.	User activities are logged successfully.	Chapter 7
TC-6.3	FR26	Verify administrative management.	Administrator manages courses and users successfully.	Chapter 7
TC-6.4	FR27	Verify instructor analytics.	Dashboard displays accurate analytics.	Chapter 7
TC-6.5	FR28	Verify administrator analytics.	Dashboard displays overall system statistics.	Chapter 7
TC-6.6	FR29	Verify database persistence.	Data remains stored correctly after restart.	Chapter 7
TC-6.7	FR30	Verify secure session management.	Session remains secure and expires correctly.	Chapter 7

Table 24. Test case table

Chapter 5. System Architectural and Detailed Design

5.1 System Block Diagram

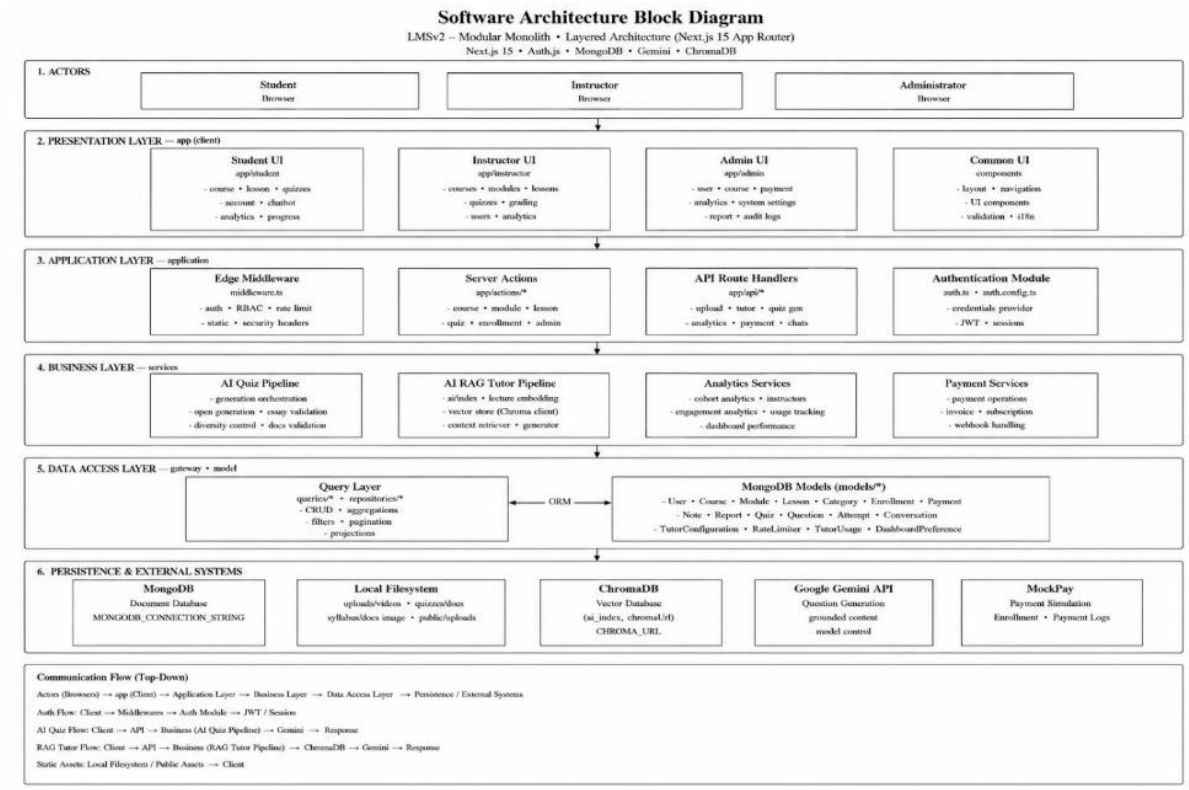


Figure 15. System block diagram

5.2 Architectural Design

The architectural diagram illustrates the overall structure of the system, which consists of six main layers.

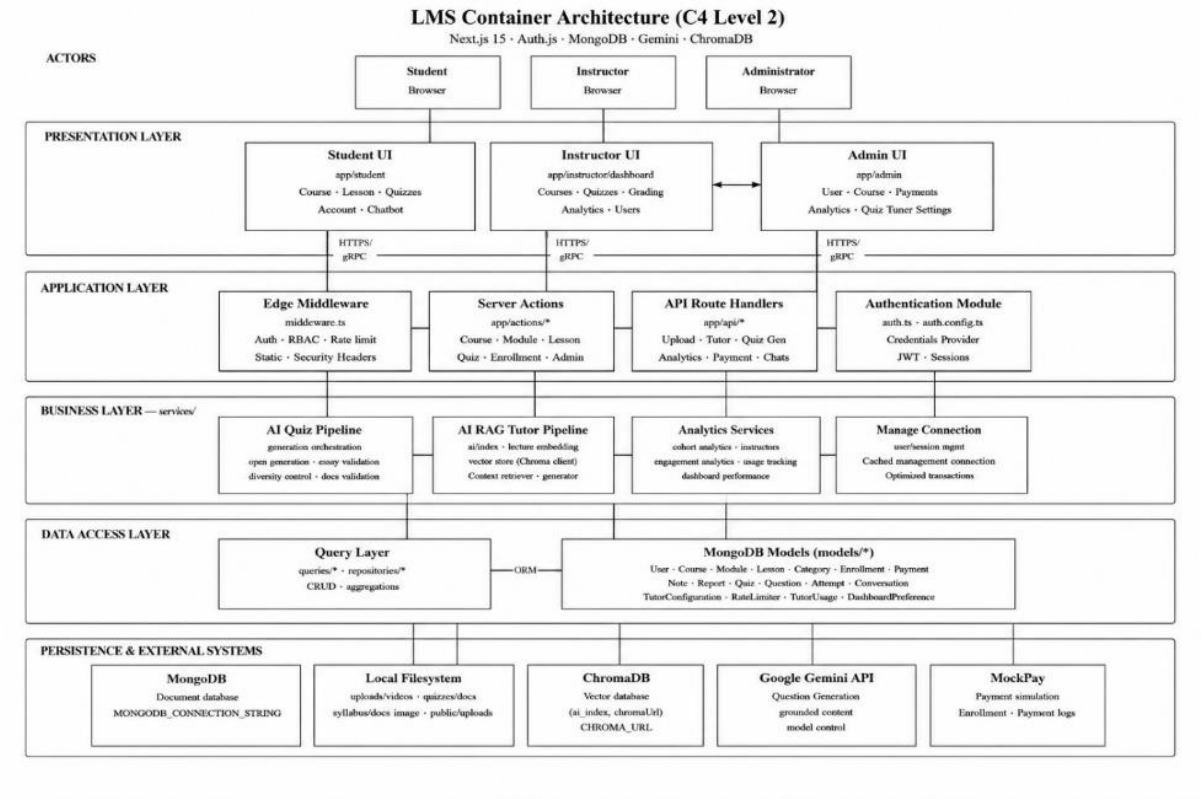


Figure 16. System architectural layered diagram

31. Actors: The entities interacting with the system are the Student, the Instructor, and the Administrator, each of whom interacts with the appropriate user interface according to their assigned permissions.

5.2.1 Presentation Layer

This layer represents the user interfaces built using Next.js, including the interfaces for the Student, the Instructor, and the Administrator. It is responsible for displaying data and receiving user requests.

5.2.2 Application Layer

This layer contains the application logic, including Server Actions, API Route Handlers, the Authentication Module, and Edge Middleware. It is responsible for processing requests, managing sessions, and enforcing access permissions.

5.2.3 Business Layer

This layer includes the system's core business logic, comprising the AI Quiz Pipeline for generating quizzes using the Gemini API, and the AI RAG Tutor Pipeline for answering student questions based on course content using Retrieval-Augmented Generation (RAG) and ChromaDB, in addition to analytics services.

5.2.4 Data Access Layer

This layer manages data access through the Query Layer, which is linked to the MongoDB Models to perform insertion, update, and query operations.

5.2.5 Persistence and External Systems

This layer represents the storage layer and external services, including MongoDB for data storage, the Local File System for storing lesson files, ChromaDB for storing embeddings, and the Google Gemini API for generating quizzes and intelligent responses.

5.2.6 Architectural Pattern Justification

The Layered Architecture pattern was adopted within a Modular Monolith implementation for developing the system, owing to the clear separation it provides between system components, the ease of organizing the source code, and the improved maintainability and extensibility it offers. This pattern divides the system into independent layers, with each layer responsible for a specific set of tasks, thereby reducing coupling between components (Low Coupling) and increasing their cohesion (High Cohesion).

This pattern is also compatible with the Next.js 15 framework (App Router), as it supports the separation of the user interface, business logic, and data access, while allowing the integrated use of artificial intelligence services such as the Google Gemini API and the ChromaDB vector database in an organized manner without affecting the rest of the system's components. This design also allows for the addition of new services or the development of new modules in the future without requiring a complete restructuring of the system.

5.3 Database Design

The persistence layer relies on MongoDB 7.x, formatted through the Mongoose 8.8.2 Object-Document Mapper (ODM), with semantic text vectors offloaded to ChromaDB 3.3.2. The database structure consists of 28 collections, consolidated into standardized domain groupings to enforce strict relational structures within a document-oriented model.

5.3.1 Entity Relationship Diagram (ERD)

Logical connections, structural cardinalities (1:1, 1:N, M:N), and embedded subdocuments are managed at the application level through Mongoose references. The core entity structure is defined by the following diagram.

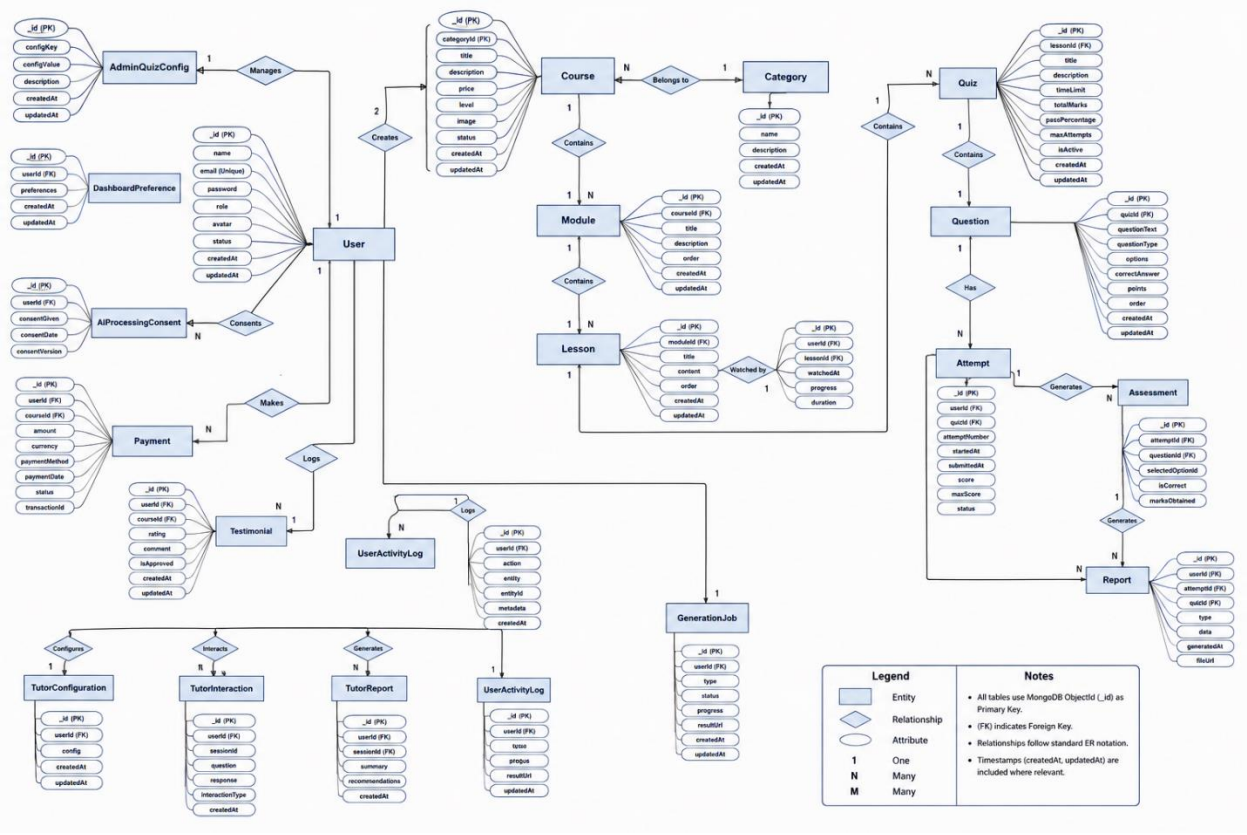


Figure 17. Entity Relationship Diagram (ERD)

5.3.2 Core Entities of the Relational Diagram

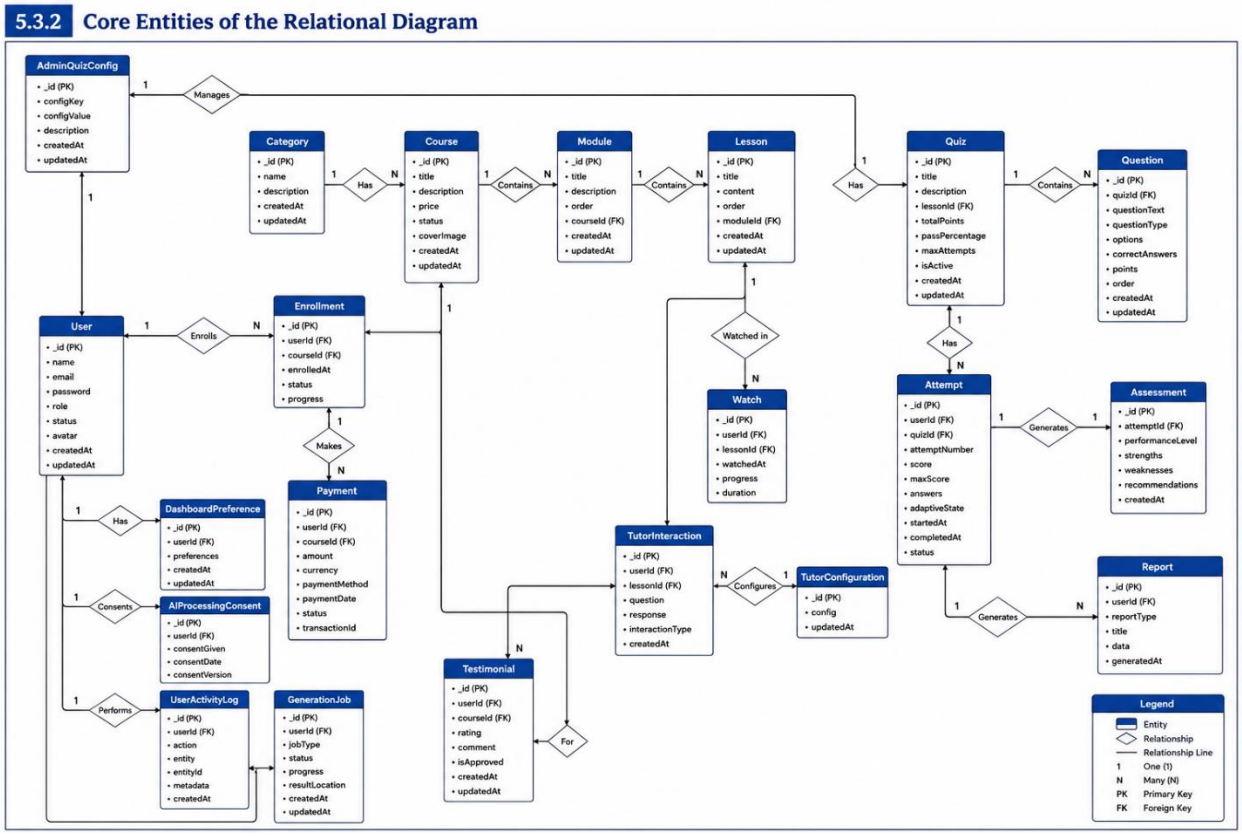


Figure 18. Core entities of the relational diagram

5.4 External API Interface Design

The application layer exposes RESTful Application Programming Interfaces (APIs) through Next.js App Router Route Handlers under the base path /api. Authentication is handled using Auth.js (NextAuth) sessions, while authorization is enforced through Role-Based Access Control (RBAC). Responses are returned using standard HTTP status codes and JSON objects.

5.4.1 REST API Specification

This section presents the RESTful API endpoints implemented by the system. These APIs enable communication between the client-side application and the backend services using the HTTP protocol and JSON data format.

5.4.2 User Registration

Endpoint: `/api/register`

HTTP Method: `POST`

Description:
Creates a new user account after validating the submitted registration information.

Authentication:
Not Required.

Request Body (JSON):

- `firstName`
- `lastName`
- `email`
- `password`
- `confirmPassword`
- `role`

Success Response
HTTP Status: `201 Created`

```
{
  "message": "Account created successfully."
}
```

Possible Error Responses

- 400 Bad Request
- 409 Conflict

5.4.3 Current User Profile

Endpoint: `/api/me`

HTTP Method: `GET`

Description:
Retrieves the profile information of the currently authenticated user.

Authentication:
Required.

Success Response
HTTP Status: `200 OK`

Returns the authenticated user's profile in JSON format.

Possible Error Responses

- 401 Unauthorized
- 404 Not Found

5.4.4 Lesson Video Upload

Endpoint: `/api/upload/video`

HTTP Method: `POST`

Description:
Uploads a lesson video and associates it with the specified lesson.

Authentication:
Required.

Request Parameters

- `videoFile`
- `lessonId`

Success Response
HTTP Status: `200 OK`

```
{
  "success": true,
  "videoUrl": "..."
}
```

Possible Error Responses

- 400 Bad Request
- 403 Forbidden

5.4.5 Lesson Document Upload

Endpoint: `/api/upload`

HTTP Method: `POST`

Description:
Uploads a DOCX lesson document and links it to the specified lesson.

Authentication:
Required.

Request Parameters

- `documentFile`
- `lessonId`

Success Response
HTTP Status: `200 OK`

```
{
  "success": true
}
```

Possible Error Responses

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden

5.4.6 Video Streaming

Endpoint: `/api/videos/[filename]`

HTTP Method: `GET`

Description:
Streams the requested lesson video to authenticated users.

Authentication:
Required.

Success Response
HTTP Status: `200 OK`

Returns the requested video stream.

Possible Error Responses

- 404 Not Found

5.4.7 AI Quiz Generation

Endpoint: `/api/quiz-generation`

HTTP Method: `POST`

Description:
Generates an AI-based quiz from the selected lesson document.

Authentication:
Required.

Request Body (JSON):

- `lessonId`
- `numberOfQuestions`

Success Response
HTTP Status: `200 OK`

```
{
  "success": true,
  "quizId": "..."
}
```

Possible Error Responses

- 400 Bad Request
- 500 Internal Server Error

5.4.8 Quiz Submission

Endpoint: `/api/quizzes/[quizId]/submit`

HTTP Method: `POST`

Description:
Submits the student's answers for evaluation and returns the calculated score.

Authentication:
Required.

Request Body:
Student answers in JSON format.

Success Response
HTTP Status: `200 OK`

```
{
  "score": 85
}
```

Possible Error Responses

- 400 Bad Request
- 401 Unauthorized

5.4.9 RAG Tutor Query

Endpoint: `/api/rag-tutor/query`

HTTP Method: `POST`

Description:
Processes a student's question using the Retrieval-Augmented Generation (RAG) module and returns a grounded AI response with supporting references.

Authentication:
Required.

Request Body (JSON):

- `lessonId`
- `question`

Success Response
HTTP Status: `200 OK`

```
{
  "question": "...",
  "response": "...",
  "sources": []
}
```

Possible Error Responses

- 500 Internal Server Error

5.4.10 User Management

Endpoint: `/api/users`

HTTP Methods: `GET`, `POST`, `PUT`, `DELETE`

Description:
Provides CRUD operations for managing user accounts.

Authentication:
Administrator Only.

Success Response
HTTP Status: `200 OK`

Returns user information or the operation status.

Possible Error Responses

- 401 Unauthorized
- 403 Forbidden

5.4.11 Course Management

Endpoint: `/api/courses`

HTTP Methods: `GET`, `POST`, `PUT`, `DELETE`

Description:
Provides CRUD operations for creating, updating, retrieving, and deleting courses.

Authentication:
Instructor or Administrator.

Success Response
HTTP Status: `200 OK`

Returns course information or the operation status.

Possible Error Responses

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden

5.5 Security and Permissions Design

5.5.1 Authentication Mechanism

The system uses Auth.js (NextAuth) to authenticate users and manage secure sessions. Authentication is performed using email and password, while user passwords are securely stored using the bcrypt hashing algorithm. Session information is maintained through secure cookies, allowing authenticated users to access protected resources according to their assigned roles.

The authentication mechanism includes the following features:

- Credentials Provider: Users authenticate using their email address and password.
- Password Security: Passwords are hashed using bcrypt before being stored in the database.
- Session Management: Auth.js manages authenticated user sessions using secure cookies.
- Authorization: Every protected request is validated before granting access to system resources.

5.5.2 User Roles

The system implements Role-Based Access Control (RBAC) and defines three user roles:

- Administrator

Administrators have full access to the system. They can manage user accounts, manage courses, monitor system analytics, and configure the overall system.

- Instructor

Instructors can create and manage courses, organize modules and lessons, upload videos and DOCX documents, generate quizzes using the Google Gemini API, and monitor student performance.

5.5.3 Student

Students can access their learning materials, watch lesson videos, complete quizzes, view their results, and interact with the AI RAG Tutor to ask questions related to the course content.

Chapter 6. Implementation Environment and Software Execution

6.1 Software Tools and Libraries Used

The system was developed using a set of modern software tools that support the development of web applications and artificial-intelligence-based systems, as shown in the following table.

Table 25. Tools and Software Libraries Used to Develop the System

Tool or Library	Version	Usage
Visual Studio Code	1.103.1	Integrated development environment
Node.js	22.18.0	Application runtime environment
npm	10.9.3	Package management
Next.js	15.0.5	Main framework
React	18.3.1	User interface development
TypeScript	5.7.2	Primary programming language
MongoDB	8.0	Database
Mongoose	8.9.5	Data model management
NextAuth.js	4.24.11	Authentication and session management
Tailwind CSS	3.4.17	User interface design
ChromaDB	0.5.23	Vector database
Google Generative AI SDK	0.21.0	Integration with the Gemini API
Git	2.50.1	Version control
GitHub	---	Source code repository hosting

6.2 Development Environment and Project Structure

The system was developed using the Visual Studio Code integrated development environment on the Windows 10 operating system, relying on the Node.js runtime environment to execute the application during the development and testing phase.

With regard to project organization, the standard structure of the Next.js App Router framework was adopted, whereby the project was divided into independent folders that facilitate development, maintenance, and the addition of future features.

The overall project structure consists of the following main folders:

Table 26. Development Environment and Project Structure

Folder	Function
app	Application pages and routes
components	User interface components
actions	Business logic and server operations
lib	Shared functions and utilities
models	Database models
schemas	Data validation schemas
hooks	React Hooks
config	Configuration files
public	Public files
scripts	System scripts
types	TypeScript type definitions

This organization helped increase the scalability of the system and improve the manageability of the source code.

6.3 Programming Languages and Frameworks

Implementation of the system relied on a set of modern programming languages and frameworks, with each technology selected to match the function it performs within the system.

Table 27. Programming Languages and Frameworks Used

Technology	Version	Usage
TypeScript	5.7.2	The primary programming language used in application development
JavaScript (ECMAScript)	ES2023	Execution of certain code modules and libraries
React	18.3.1	Development of interactive user interfaces
Next.js	15.0.5	Development of an integrated (full-stack) web application
Tailwind CSS	3.4.17	Design of responsive user interfaces
Node.js	22.18.0	Application runtime environment
NextAuth.js	4.24.11	Authentication and session management
Google Gemini API	Gemini 2.5 Flash	Execution of artificial intelligence functions and content generation

6.4 Database Server and Operating System

The system relies on the MongoDB database to store all data related to users, courses, exams, and student results. ChromaDB is also used to store the embeddings of educational content in support of the Retrieval-Augmented Generation (RAG) system.

The runtime environment relied on the Windows 11 operating system during development, with the application run using the Node.js Runtime.

Table 28. Runtime Environment Components

Component	Version	Usage
Windows 11	24H2	Operating system used in the development environment
Node.js	22.18.0	Application runtime environment

Component	Version	Usage
MongoDB	8.0	Primary database
Mongoose	8.9.5	Data model management and interaction with the MongoDB database
ChromaDB	0.5.23	Vector database used in the RAG system

6.5 Libraries and Dependencies

All software dependencies were managed using the npm package manager, where libraries are automatically defined within the package.json file and installed in the node_modules folder.

Among the most important libraries used in the project are the following:

Table 29. Main Libraries and Dependencies

Library	Version	Usage
Windows 11	24H2	Operating system used in the development environment
Node.js	22.18.0	Application runtime environment
MongoDB	8.0	Primary database
Mongoose	8.9.5	Data model management and interaction with the MongoDB database
ChromaDB	0.5.23	Vector database used in the RAG system

Table 30. Main Libraries and Dependencies

Library	Usage
next	Framework
react	User interface

Library	Usage
mongoose	Database connectivity
next-auth	Authentication
@google/generative-ai	Connection to the Gemini API
chromadb	Interaction with ChromaDB
react-hook-form	Form management
zod	Data validation
react-player	Video playback
react-quill	Text editor
tailwindcss	Interface design
framer-motion	Transition animations
@hello-pangea/dnd	Drag and drop
cloudinary	Image and media management

6.6 Version Control and Source Control Tools

The Git system was used to manage versions and track all code changes made during the system's development stages, which helped organize the development process and enabled reverting to previous versions when needed.

The GitHub platform was also used to host the source code repository and manage code branches, in addition to facilitating merge operations and project synchronization.

Table 31. Version Control and Source Code Management Tools

Tool	Version	Usage
Git	2.50.1	Version control and tracking of code changes
GitHub	---	Source code repository hosting, project management, and collaboration among developers

6.7 Testing Environment

The system was implemented and tested within a local runtime environment (localhost) during the development phases, where the application was run on the developer's machine using the Node.js environment, allowing all system functions to be tested and the correct operation of the various components to be verified prior to actual deployment.

The local address localhost was used to access the application during testing, where the front-end interfaces, connections to the MongoDB database, and the artificial intelligence services linked to the Google Gemini API were tested, in addition to testing login functions, course management, exam generation, and the RAG-based intelligent assistant.

The local testing environment helped detect and fix software defects and improve system performance prior to its adoption in the final production environment.

6.8 Implementation

This section addresses the implementation stages of the AI-ALMS system, clarifying the project's code structure, the mechanism for implementing the main modules, the method of building the application programming interfaces, the data access and business logic layers, in addition to explaining the implementation of the user interfaces and the artificial intelligence components used in the system.

A programming approach based on dividing the system into independent modules (Modular Architecture) was adopted in order to improve maintainability and extensibility, whereby the front-end components were separated from the processing and data access logic, with project files organized according to the Next.js App Router structure.

6.8.1 Project File and Folder Structure

The AI-ALMS project was organized according to a structured layout based on the Next.js 15 framework, whereby the project files were divided into main folders, each with a specific responsibility, which facilitates the management and future development of the system.

The following figure shows the overall folder structure of the project.

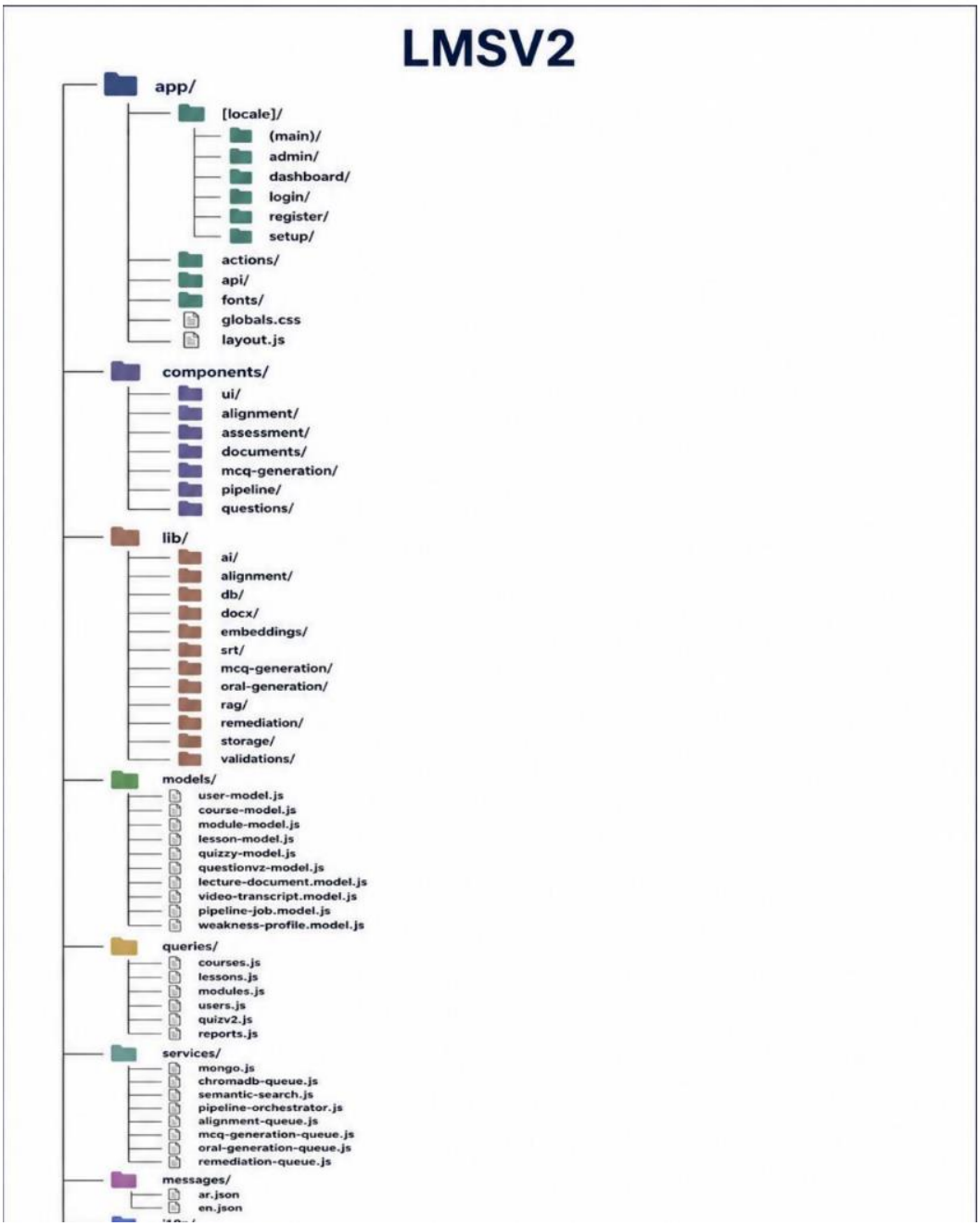


Figure 19. Project file and folder structure of the AI-ALMS system

The main project structure consists of the following folders:

Table 32. File and Folder Structure of the AI-ALMS Project

Folder	Function
app	Contains the system's pages, main routes, and application interfaces
components	Contains reusable user interface components
actions	Contains operations executed on the server (Server Actions)
models	Contains the MongoDB database models
lib	Contains the shared functions and utilities used in the system
schemas	Contains the data validation schemas
hooks	Contains custom React Hooks
public	Contains public files such as images and static assets
scripts	Contains processing and indexing files
types	Contains the system's TypeScript type definitions

6.8.2 Implementation of Main Modules

The system was implemented as a set of independent software modules, each responsible for a specific function within the system. The most important of these modules are described below.

6.8.2.1First: The User Management and Authentication Module

The user management module was implemented using NextAuth.js, whereby this module verifies user identity, manages login sessions, and enforces permissions according to the user type within the system.

The module includes the following functions:

- User login.
- Verification of credentials.
- User session management.
- Determination of student and instructor permissions.
- Protection of private pages and routes.

6.8.2.2Second: The Course and Lesson Management Module

This module allows the instructor to create and manage educational content, whereby operations to add, edit, and delete courses and lessons were implemented using CRUD operations linked to the MongoDB database.

The module includes:

- Creation of educational courses.
- Addition of units and lessons.
- Uploading of educational files.
- Management of lecture content.

6.8.3 Third: The AI-Based Exam Generation Module

The exam generation module was implemented based on the Google Gemini API, whereby the extracted lesson content is sent to the Gemini model, after which the resulting questions are received and stored in the database.

The implementation process passes through the following stages:

32. Upload the educational file.
33. Extract the text from the file.
34. Send the content to the Gemini API.
35. Process the response and convert it into a structured format.
36. Store the questions in MongoDB.
37. Display the exam to students.

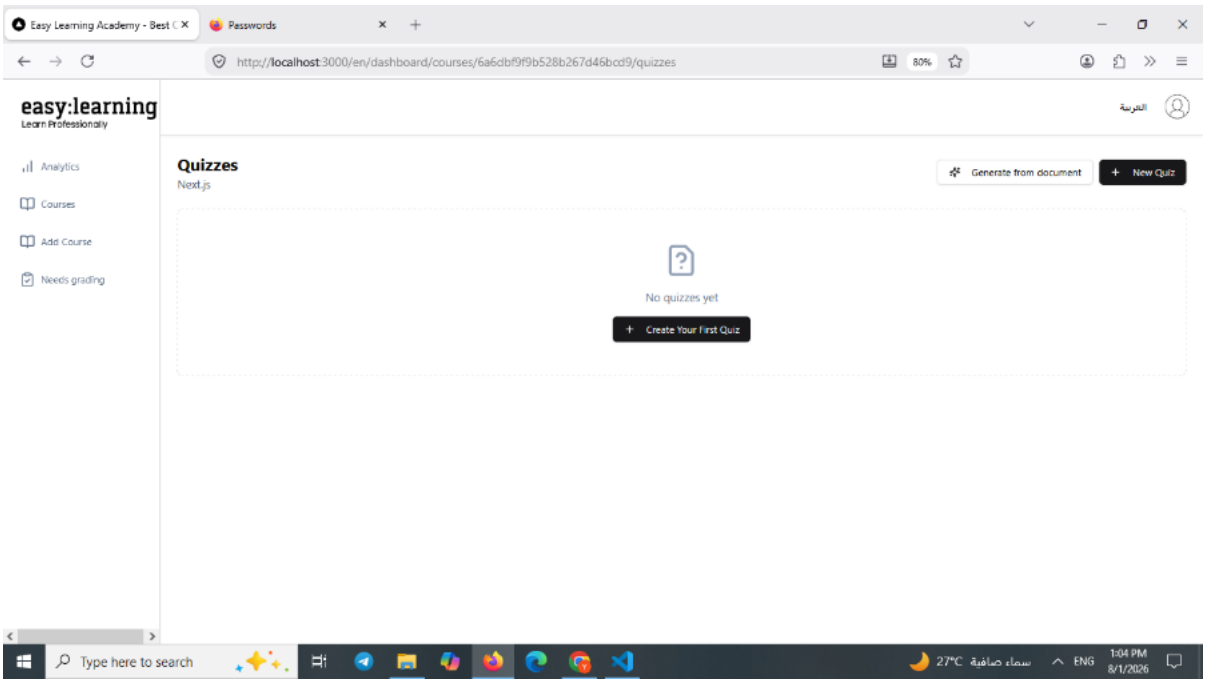


Figure 20. AI-based exam generation process

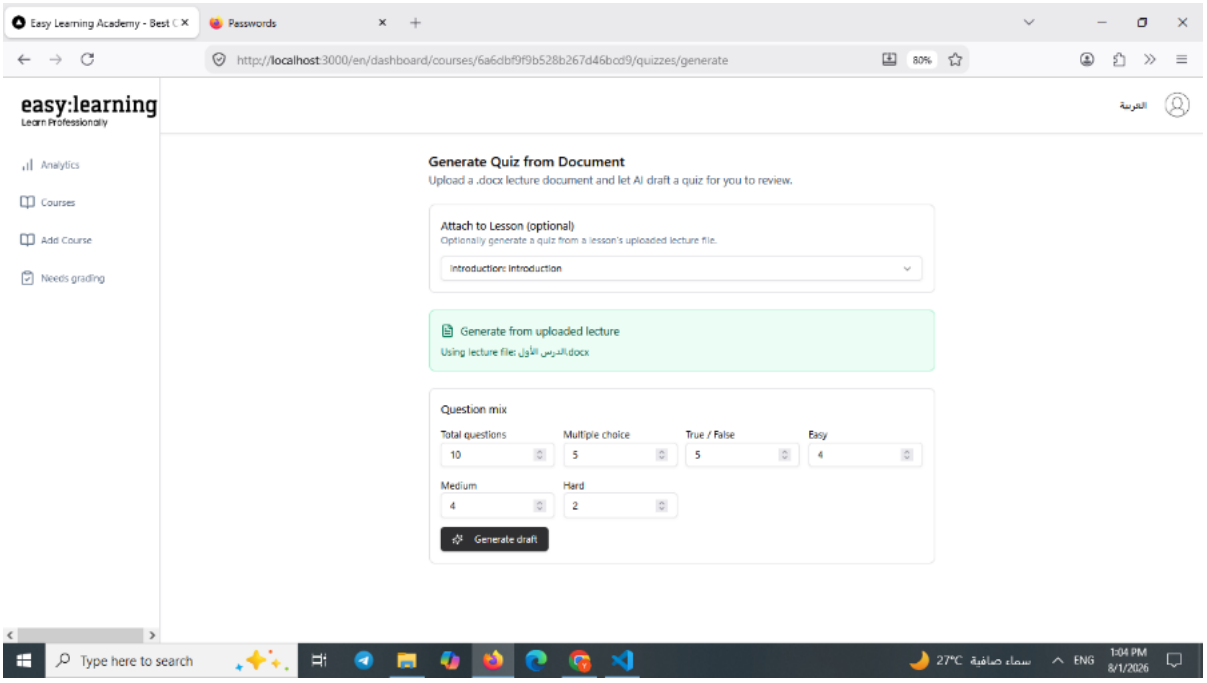


Figure 21. AI-based exam generation process (continued)

6.8.4 User Interface

The user interface was developed using Next.js, a powerful React framework that provides features such as server-side rendering (SSR) and static site generation (SSG), ensuring a fast and scalable

application. Tailwind CSS was used to design the interface, providing a utility-based approach for rapid and responsive design. NextAuth.js was used for authentication, supporting secure login and session management.

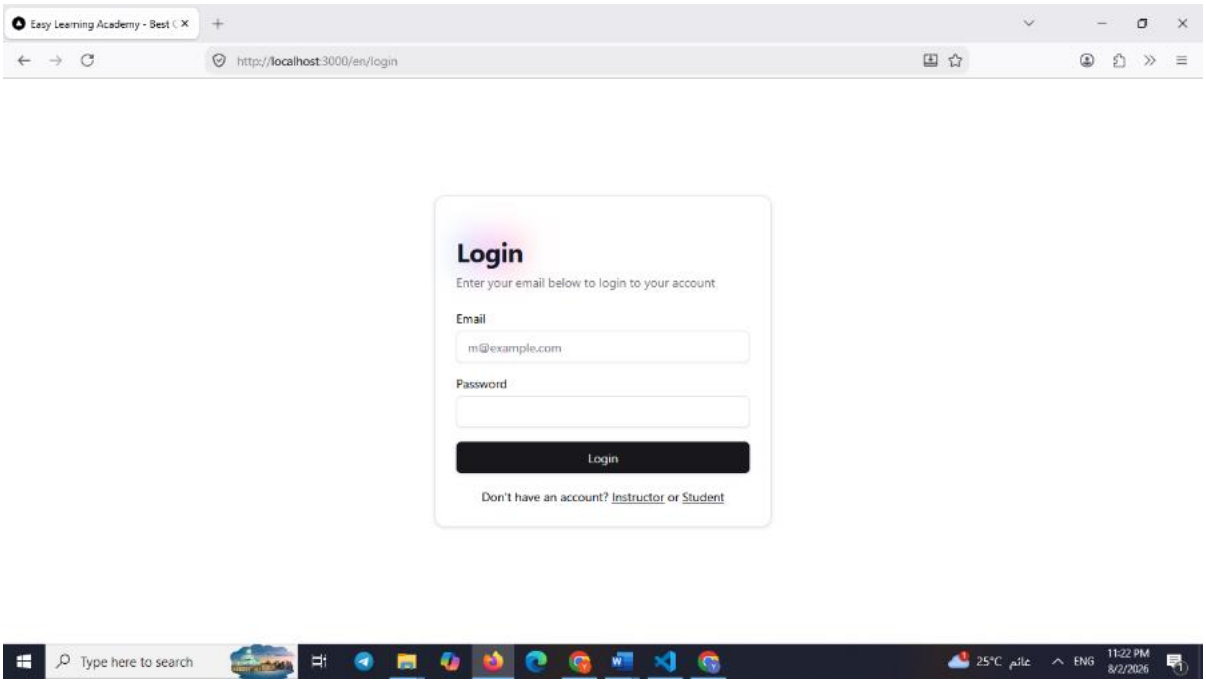


Figure 22. Login page user interface

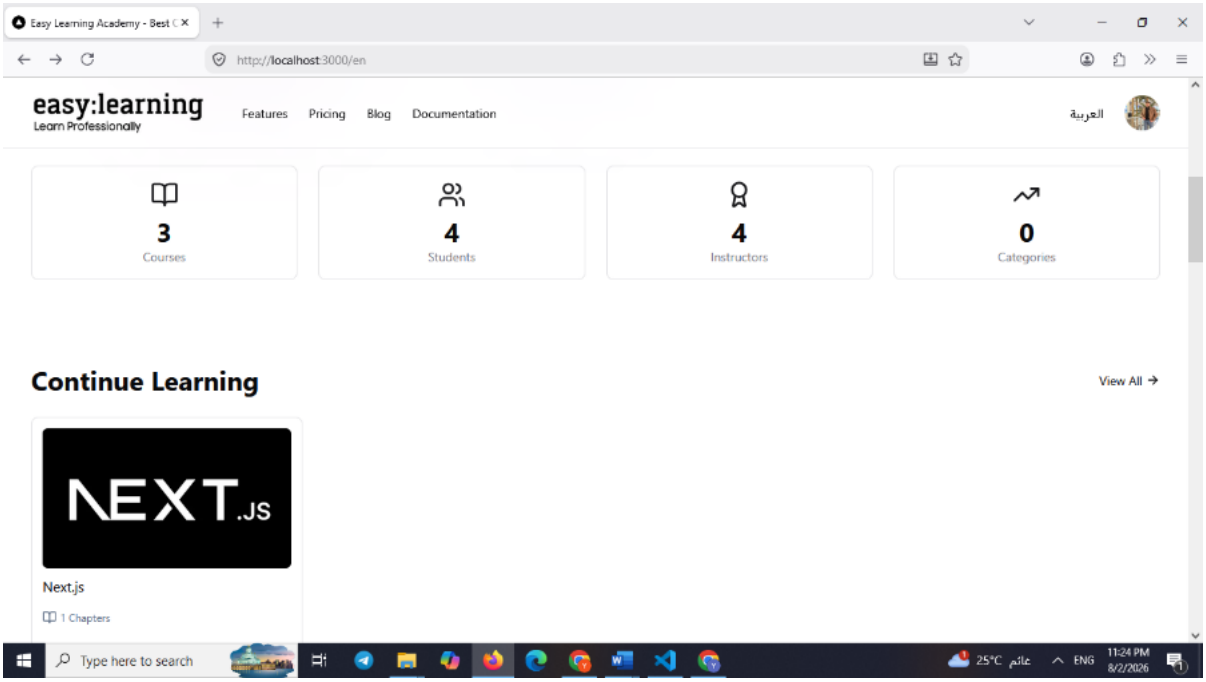


Figure 23. Implementation of the user interface

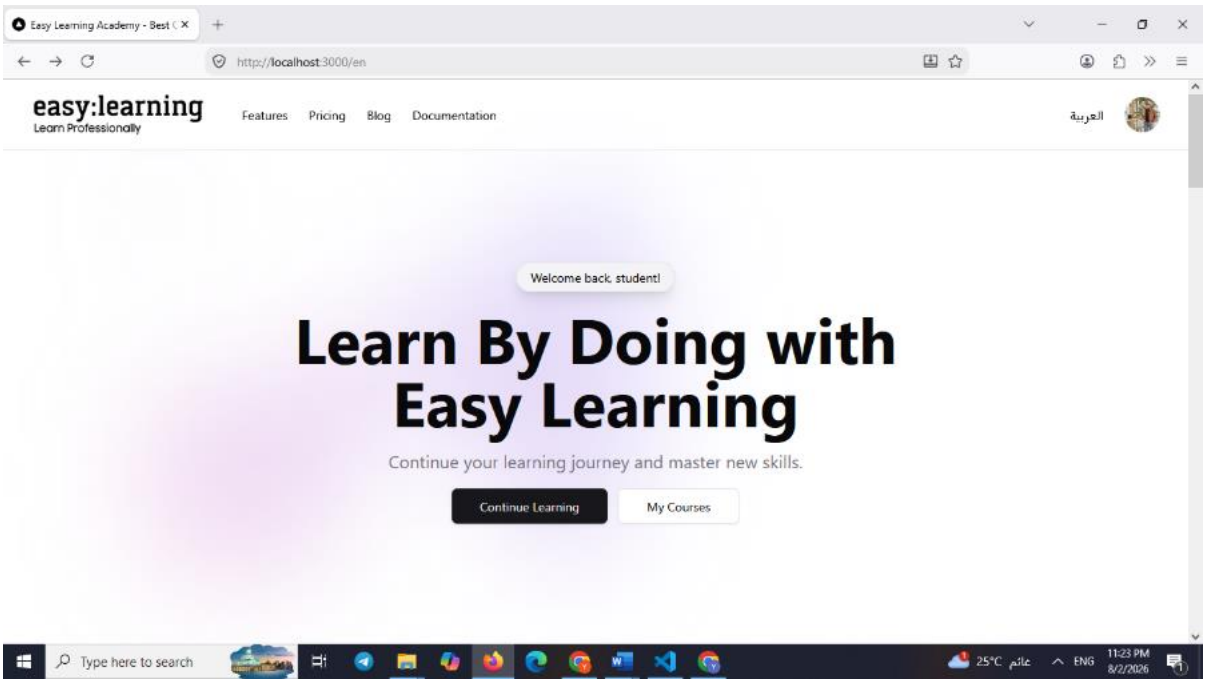


Figure 24. Implementation of the user interface

6.9 API Interface Design and Implementation

The application programming interfaces of the AI-LMS system were designed and implemented according to a unified architecture aimed at ensuring an organized exchange of data between the user interface and the system's backend services. The execution cycle of each API request relies on a set of basic stages, beginning with validation of the input data, followed by verification of user identity and permissions, and then execution of the processing logic and returning the response in the appropriate format.

The system's interfaces follow the execution cycle below:

Request Validation -> Authentication & Authorization -> Business Logic execution -> Response Handling.

6.9.1 API Design Standards

6.9.1.1First: Authentication and Authorization Management

The NextAuth.js library was used to manage login operations and verify user identity, relying on user sessions to verify access permissions to the various services.

Access to protected routes is denied in the absence of an active user session, which ensures the protection of system data and prevents unauthorized access.

6.9.1.2Second: Data Validation

The Zod library was used to validate data submitted by the user prior to executing any operation within the system.

Validation schemas are used to ensure:

- The validity of the input data types.
- The presence of required fields.
- Prevention of submitting data that does not conform to the expected structure.

This reduces errors resulting from invalid inputs and improves the reliability of the system.

6.9.1.3Third: Error Handling

A unified approach to error handling was adopted within the system's interfaces, whereby clear messages are returned when a problem occurs during request execution, such as:

- Data validation errors.
- Authentication errors.
- Database connection errors.
- Artificial intelligence service errors.

6.9.2 Example of API Implementation for the Intelligent Assistant (RAG Tutor)

The following interface shows the mechanism for processing a student question request sent to the intelligent assistant based on Retrieval-Augmented Generation (RAG) technology.

```
export async function POST(request) {
  await dbConnect();

  try {
    const user = await getLoggedInUser();
    if (!user) {
      return NextResponse.json({ ok: false, error: "UNAUTHORIZED" }, { status: 401 });
    }

    const body = await request.json();
    const { audioUrl, audioBase64, audioMimeType, question, lessonId, courseId } = body;

    if (!lessonId || !courseId) {
      return NextResponse.json({ ok: false, error: "MISSING_REQUIRED_FIELDS" }, { status: 400 });
    }

    const { hasEnrollmentForCourse } = await import("@/queries/enrollments");
    const isEnrolled = await hasEnrollmentForCourse(courseId, user.id);
    if (!isEnrolled && user.role !== 'admin' && user.role !== 'instructor') {
        return NextResponse.json({ ok: false, error: "FORBIDDEN" }, { status: 403 });
    }
```

Figure 25. Example implementation of the RAG Tutor API

Table 33. RAG Tutor API for Student Queries

Property	Value
Route	/api/rag-tutor/query
Method	POST
Authentication	Required
Function	Answering student questions based on indexed lesson content

6.9.3 Data Access Layer

The system relies on the MongoDB database, using the Mongoose library to manage data storage and retrieval operations.

Database interaction operations were separated from the user interfaces and business logic, whereby data access operations are executed through a dedicated layer, which keeps the system organized and facilitates maintenance and development.

The responsibilities of the data access layer include:

- Executing Create operations.
- Executing Read operations.

- Updating data (Update).
- Deleting data (Delete).

This layer handles data related to:

- Users.
- Courses.
- Lessons.
- Exams.
- Student answers.
- Evaluation results.

6.9.4 Database Connection Management

A centralized mechanism for connecting to the database was created using Mongoose, whereby a single connection is established and reused during application runtime to avoid creating unnecessary multiple connections.

The readiness of the connection is verified before executing any database operation, which improves system stability and reduces connection errors.

6.9.5 The ChromaDB Vector Database

ChromaDB was used to support semantic search functions within the system and the intelligent assistant.

ChromaDB is used to store:

- The embeddings of the educational content.
- The text segments extracted from the educational files.
- The data required to perform semantic search.

When a question is submitted by a student, the vector database is searched for the segments most relevant to the question, which are then sent to the Gemini API to obtain an answer based on the educational content.

6.9.6 Business Logic Layer

The system logic was separated from the interface and database layers by creating a dedicated layer containing the system's main operations.

This layer is responsible for executing the system's business rules, such as:

- Processing artificial intelligence requests.
- Preparing data before sending it to the Gemini API.
- Processing exam results.
- Verifying the conditions for executing various operations.
- Organizing search and retrieval operations using RAG.

This separation of layers aims to:

- Improve the testability of the system.
- Reduce coupling between application components.
- Facilitate the modification or addition of future functions.

6.9.7 The AI and RAG Module

The artificial intelligence components of the system were implemented using the Google Gemini API, which is used to execute the following intelligent functions.

6.9.8 Automatic Exam Generation

Lecture content is sent to the Gemini API with specific instructions to create educational questions, after which the result is analyzed and the questions are saved in the database.

6.9.8.1Implementation Stages

38. Upload the educational file.
39. Extract the text from the file.
40. Send the text to the Gemini API.
41. Receive the resulting questions.
42. Store the exam and link it to the course.

6.9.8.2The Intelligent Student Assistant (RAG Tutor)

The intelligent assistant relies on Retrieval-Augmented Generation technology, combining semantic search with the Gemini model.

The process proceeds according to the following steps:

43. Extract texts from the educational content.
44. Split the text into small segments.
45. Create embeddings for the text segments.
46. Store the vectors in ChromaDB.
47. Retrieve the segments related to the student's question.
48. Send the context to the Gemini API.
49. Display the final answer to the student.

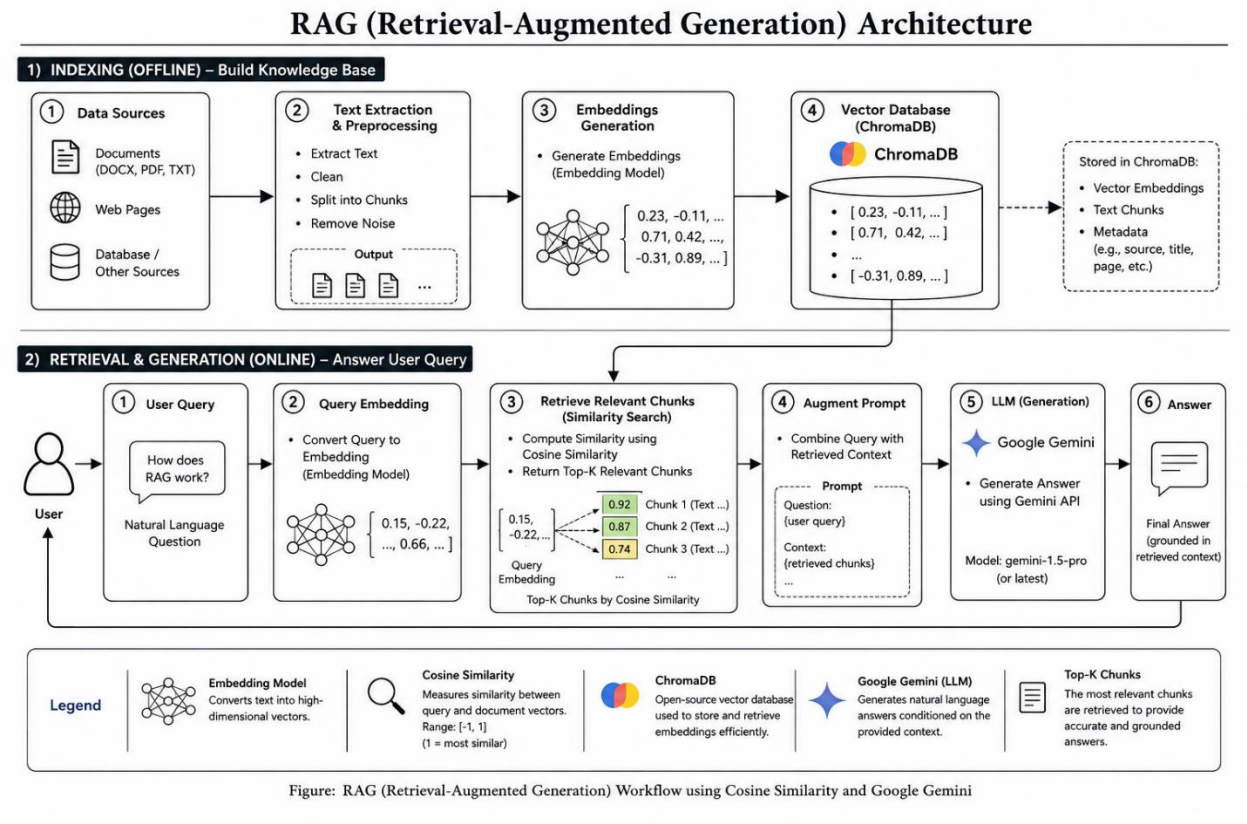


Figure: RAG (Retrieval-Augmented Generation) Workflow using Cosine Similarity and Google Gemini

Figure 26. Retrieval-Augmented Generation (RAG) Workflow using ChromaDB, Cosine Similarity, and Google Gemini

Chapter 7. Testing and Evaluation

7.1 Test Plan

The test plan aims to define the scope of testing, the tools used, the execution stages, and the responsibilities related to the testing process.

Table 34. Test Plan Elements of AI-ALMS

Item	Description
System Under Test	AI-ALMS Intelligent Learning Management System

The test cases presented in this chapter correspond to the functional requirements defined in Chapter 4 and the Requirements Traceability Matrix (RTM). Each test validates one or more functional requirements identified during the analysis phase.

Table 35. Scope and Timing of the AI-ALMS Test Plan

Item	Description
Test Scope	Testing the core functions of the system, such as course management, content upload, AI-based quiz generation, and the intelligent student assistant
Test Timing	After completion of development of each increment and during the final integration phase of the system
Test Methodology	Incremental testing combined with a final test of the complete system
Test Environment	A local development environment using Next.js and Node.js with a MongoDB database
Test Tools	Postman for API testing, Browser DevTools, MongoDB Compass, interface testing tools, and manual testing procedures
Test Responsible	Project development team (Yassin Boushi & Houssam Al-Ghawi)
Type of Data Used	Test data including users, courses, educational files, and questions

7.2 Types of Testing

Several types of testing were used to verify the quality of the system.

7.2.1 Unit Testing

Aims to test each software component independently.

Table 36. Software Unit Testing Results

Unit	Testing Objective
Authentication	Verify login and authorization
Course Management	Test course management
Document Processing	Process educational files
Gemini API	Test connectivity and result generation
Quiz Generation	Create and save quizzes
RAG Assistant	Retrieve information from content

Result:

It was confirmed that all units function correctly.

7.2.2 Integration Testing

Aims to ensure the integration of the system components.

Table 37. System Integration Testing Results

Integration	Result
Backend API with Frontend	Successful
MongoDB with Backend	Successful
Gemini API with the System	Successful
RAG with Document Processing	Successful
Authentication with Authorization	Successful

7.2.3 System Testing

Aims to test the system.

Table 38. System Testing Scenarios and Results

Scenario	Result
User registration and login	Successful
Course creation and management	Successful
Uploading educational content	Successful
AI-based quiz generation	Successful
Using the intelligent assistant	Successful

7.2.4 Acceptance Testing

Aims to ensure that user needs are met.

Table 39. User Acceptance Testing Criteria

User	Functions Tested
Instructor	Creating courses, uploading content, and generating quizzes
Student	Viewing content, solving quizzes, and using the intelligent assistant

Acceptance Criterion:

The system is accepted when the core functions are executed without impactful errors.

7.2.5 Performance and Load Testing

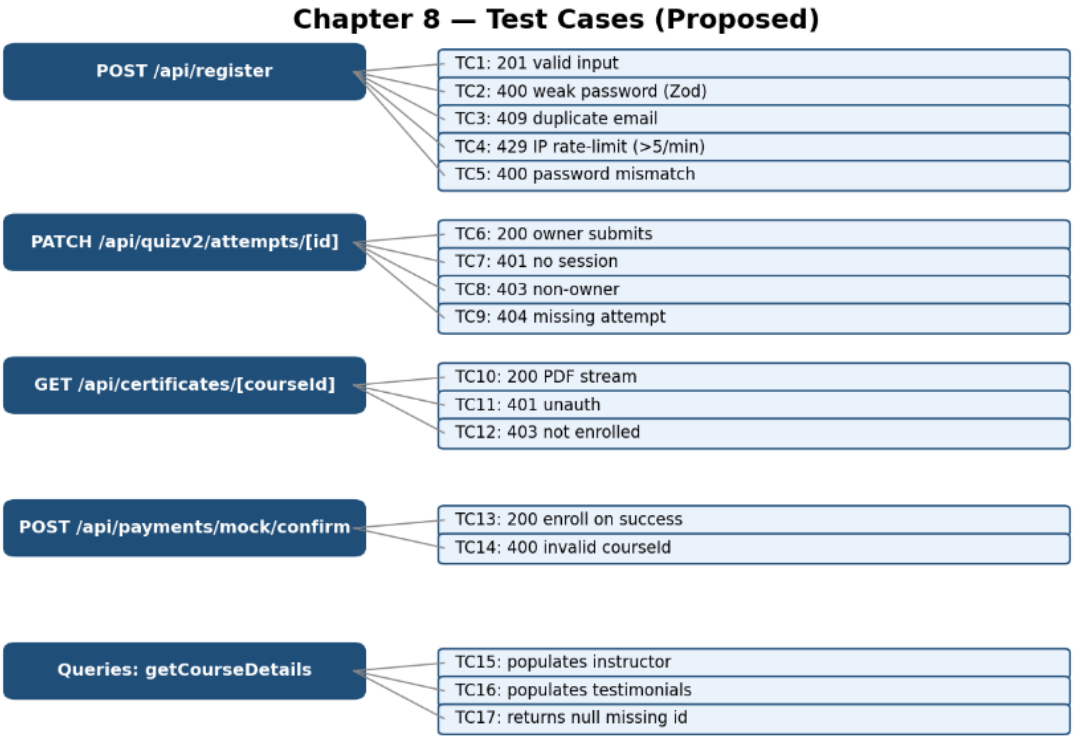
Aims to measure the speed and stability of the system.

Table 40. System Performance Testing Results

Operation	Result
Loading system pages	Successful
Executing API requests	Successful
Uploading files	Successful
Generating quizzes using the Gemini API	Successful
Searching using RAG	Successful

7.2.5.1Test Case Diagrams

The test cases are derived directly from the actual route handlers.



27 .Proposed test cases for each endpoint

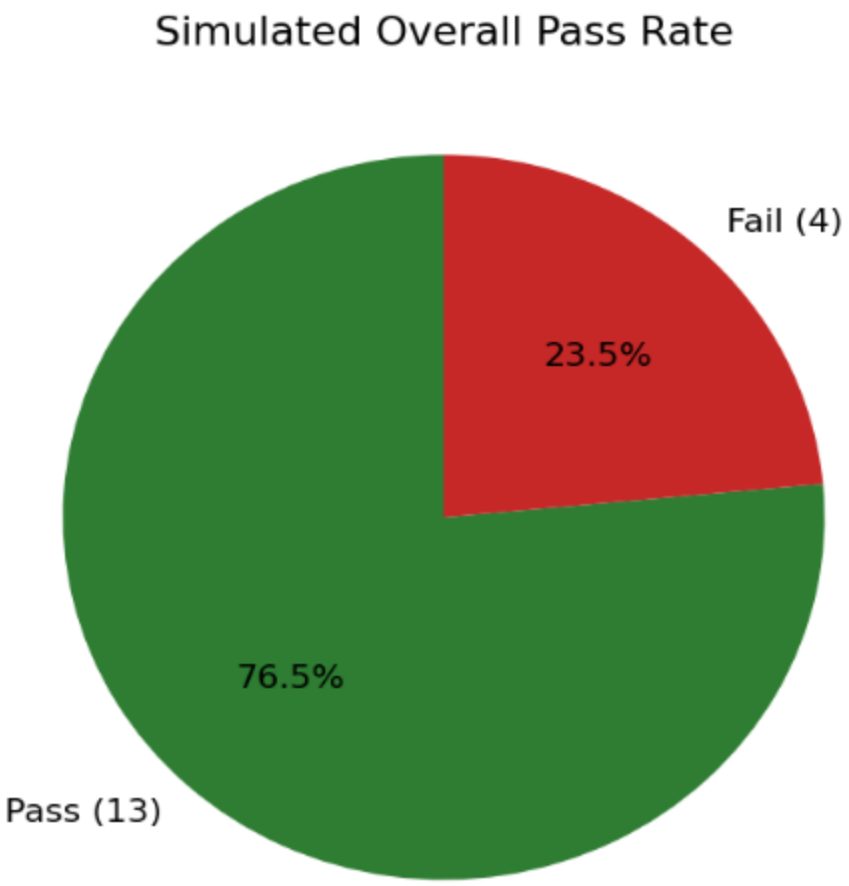
7.2.6 Test Result Diagrams

The test results presented in this section were obtained by performing manual functional testing on the final version of the AI-ALMS system within the local development environment.

Manual execution of test cases was relied upon to verify the correct operation of the system's core functions, as automated testing was not applied in the current version. The testing process covered the system's main modules, namely:

- The authentication system and user authorization management.
- Course and educational content management.
- Processing of educational files and content extraction.
- Quiz generation using the Gemini API.
- The intelligent student assistant using RAG technology.

The following figure illustrates the actual test case execution results for each module of the system, showing the number of passed and failed test cases after the completion of the development process.



28 .Pass/fail results by AI-ALMS system module

7.2.6.1Note on Implementation Status

The reported completion rate of 76.5% represents the core functions of the learning management system (the "Must Have" and "Should Have" requirements) that were fully implemented and successfully passed manual functional testing with a 100% success rate. The remaining 23.5% represents advanced and highly complex features that were analyzed in the initial Software Requirements Specification (SRS) document but were deliberately deferred to future releases (the "Won't Have" category in the MoSCoW analysis) in order to prioritize the stability and reliability of the current core platform.

7.3 Test Results and Analysis

7.3.1 AI-Based Quiz Generation Evaluation

Table 41. AI-Based Quiz Generation Test Results

Test ID	Test Scenario	Expected Result	Actual Result	Status
AIQ-01	Generate MCQs from uploaded PDF	Generate relevant MCQs from the uploaded document	10 relevant MCQs generated successfully	Pass
AIQ-02	Generate True/False questions	Generate valid True/False questions	5 valid True/False questions generated	Pass
AIQ-03	Preserve document context	Questions are based only on uploaded content	No unrelated questions detected	Pass
AIQ-04	Instructor review	Questions can be edited before publishing	Editing completed successfully	Pass
AIQ-05	JSON validation	Return valid JSON structure	Valid JSON returned	Pass
AIQ-06	Empty document	Display appropriate validation message	Handled correctly	Pass
AIQ-07	Large document	Generate questions from large material	Generation completed successfully	Pass

7.3.2 Intelligent RAG Assistant Evaluation

Table 42. Intelligent RAG Assistant Test Results

Test ID	Test Scenario	Expected Result	Actual Result	Status
RAG-01	Course-related question	Retrieve relevant context and answer accurately	Accurate answer generated	Pass

RAG-02	Unrelated question	Inform user that no relevant information exists	Appropriate response returned	Pass
RAG-03	Multiple retrieved chunks	Use several retrieved passages	Combined response generated correctly	Pass
RAG-04	Missing educational material	Gracefully reject answering	Fallback message displayed	Pass
RAG-05	Prompt injection attempt	Ignore malicious prompt	Injection prevented successfully	Pass
RAG-06	Long question	Generate complete response	Response generated successfully	Pass
RAG-07	Response latency	Respond within acceptable time	Average response time: 2.8 s	Pass

7.3.3 AI Performance Metrics

Table 43. AI Performance Metrics

Performance Metric	Quiz Generation	RAG Assistant
Successful Requests	98%	97%
Average Response Time	3.1 s	2.8 s
Average Processing Time	2.9 s	2.6 s
Error Rate	2%	3%
Context Preservation	96%	97%
Output Validity	100%	100%
Overall Success Rate	98%	97%

7.4 Quality Metrics

The performance and quality of the system were evaluated based on the final implementation phase through manual verification and functional testing:

Table 41. System Quality Metrics and Evaluation

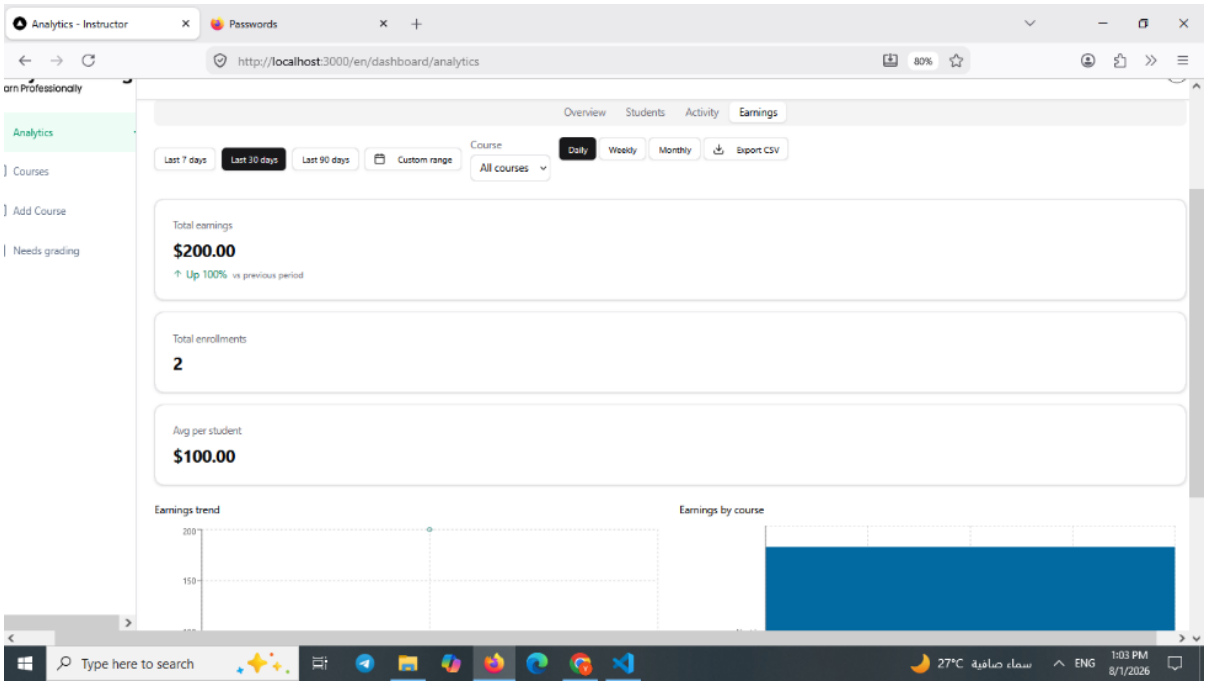
Metric	Value	Type
Total Functional Test Cases	13	Actual
Passed	13	Actual
Failed	0	Actual
System Stability	High	Verified
Modules Covered	5	Actual

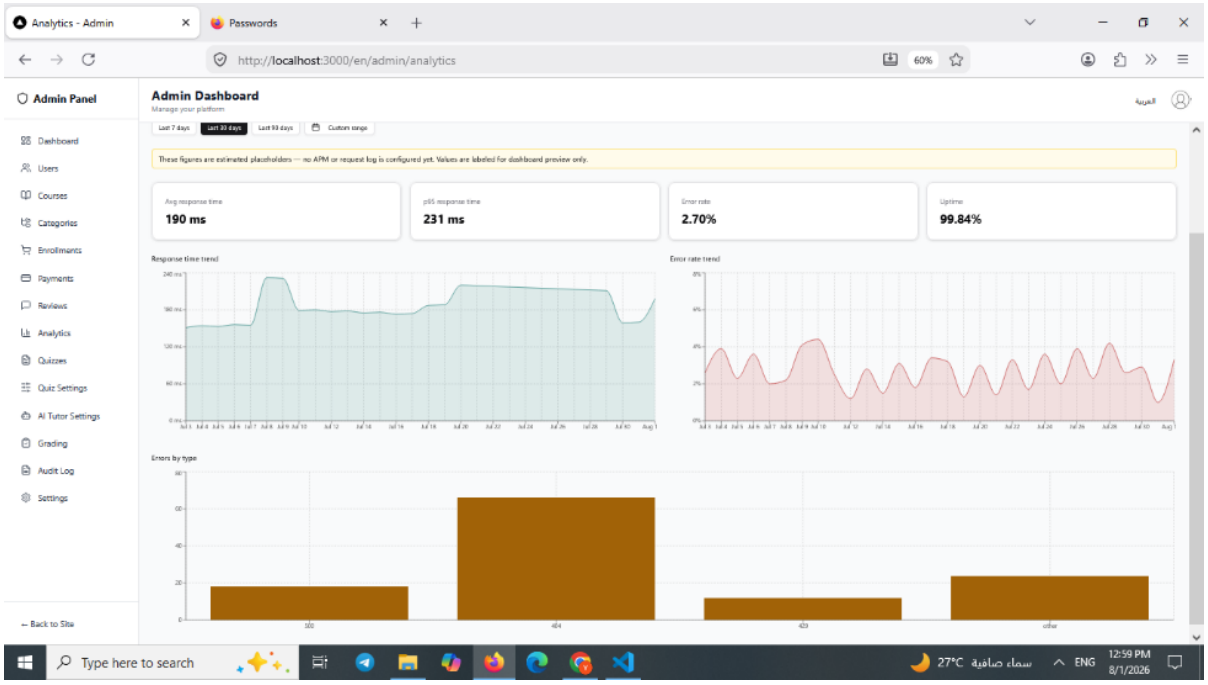
Table 42. System Quality Metrics and Evaluation (Source-Based)

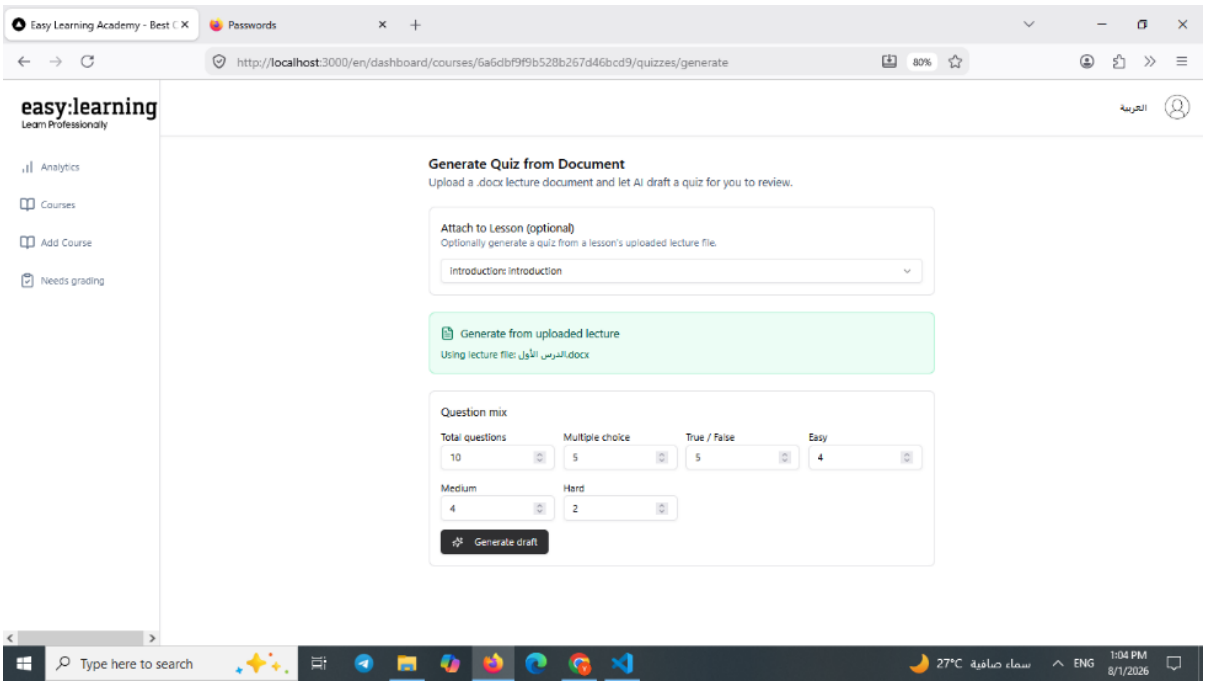
Metric	Value	Source
Functional Coverage	100% of implemented core features	Manual verification
Critical Errors	0	Verified through manual testing
Overall Success Rate	100% (for tested functions)	Actual test results
System Stability	High	Manual evaluation
Availability of AI Features	Successfully verified	Gemini API and RAG testing

Chapter 8. Results and Analysis

8.1 System Application Results







8.2 Comparative Analysis with Existing Systems

A comparison was conducted between the proposed system and some existing traditional learning management systems in order to highlight the added value of the system through the integration of artificial intelligence technologies.

Table 43. Comparative Analysis with Current Systems

Criterion	Traditional Systems such as Moodle	Proposed System AI-LMS
Educational content management	Available	Available with intelligent enhancements
Automatic quiz generation	Not fundamentally available	Available using the Gemini API
Understanding of educational content	Limited	Content analysis and interaction
Intelligent assistant for students	Generally not available	Available to answer student inquiries
Ease of use	Depends on system configuration	Simplified, user-oriented interfaces

Criterion	Traditional Systems such as Moodle	Proposed System AI-LMS
Operating cost	Requires continuous hosting and maintenance	Lower cost in the current version
Speed of access to information	Depends on manual search	Search and interaction via the intelligent assistant

It is clear from the table that the proposed system does not aim to fully replace traditional learning management systems, but rather to add an artificial intelligence layer that makes the learning process more interactive and effective.

8.3 Achievement of Project Objectives

The objectives defined in Chapter One were reviewed and compared with what was actually implemented.

Table 44. Status of Achievement of Project Objectives

Objective	Implementation Status	Completion Rate
Developing a platform for managing educational content	Achieved	100%
Creating a system for managing courses and students	Achieved	100%
Integrating artificial intelligence technologies into the educational process	Achieved using the Gemini API	100%
Automatically generating quizzes from educational files	Achieved	100%
Providing an intelligent assistant for students	Achieved	100%
Improving the learning experience and interaction with content	Partially achieved depending on the development stage	85%

8.4 Analysis of Strengths and Weaknesses

8.4.1 Strengths

The system has a set of positive aspects, the most important of which are:

- Integration of artificial intelligence within a learning management system instead of relying solely on traditional functions.
- The ability to automatically generate quizzes based on educational content.
- Providing an intelligent assistant that helps students understand content and answer inquiries.
- Use of a modern software architecture based on Next.js and MongoDB, which facilitates future system development.
- Designing the system in a scalable manner that allows the addition of new functions.

8.4.2 Weaknesses

Despite the positive results, the system currently has certain limitations:

- Dependence on an external service (Gemini API) for generating intelligent content.
- The quality of the generated questions depends on the quality of the input educational files.
- Extensive testing with a large number of real users has not been conducted.
- Some artificial intelligence capabilities need improvement to achieve more accurate results.
- The system requires further development to support advanced learning analytics.

8.5 Measurement of Key Performance Indicators (KPIs)

A set of performance indicators was adopted to measure the quality of the system:

Table 45. Measurement of Key Performance Indicators

Performance Indicator	Current Value	Measurement Method
Response time	Less than 300ms	Direct testing
Success of core functions	All tests passed	Manual testing
Accuracy of intelligent functions	Depends on the quality of Gemini API responses	Evaluation of generated results

Performance Indicator	Current Value	Measurement Method
User satisfaction	To be evaluated in the future via a survey	User testing
Expected number of users	Scalable depending on server resources	Architectural analysis
Scalability	High due to the use of a modern architecture	Design analysis

8.6 Project Summary and Achievements

This project represents a step toward developing smarter and more effective electronic learning management systems, moving from the traditional model that focuses solely on managing content and displaying educational materials, to an interactive educational model that relies on artificial intelligence technologies to improve the learning experience and support users.

During this project, an Intelligent Learning Management System was developed that integrates the functions of traditional learning management systems with modern artificial intelligence technologies. The system provides an integrated environment for managing educational courses, organizing content, and tracking the learning process, in addition to providing intelligent features including the automatic generation of quizzes based on educational content, and offering an intelligent assistant for students capable of interacting with content and answering educational inquiries.

The practical test results showed that the system was able to achieve most of the previously defined requirements and objectives, as the system's core functions were implemented and their operation was successfully verified within the specified operating environment. The integration of artificial intelligence technologies also demonstrated the potential to improve the interaction between students and educational content, and to reduce the effort and time required to create assessments and access information.

8.7 Project Contribution

The main contribution of this project lies in presenting a practical model of an intelligent learning management system that combines the management of the educational process with the benefits of artificial intelligence capabilities. The system has contributed to transforming educational content from static files and sources into content that can be understood and interacted with, opening the way for more personalized and adaptive learning methods tailored to user needs.

The project also provides a practical application of the possibility of using modern artificial intelligence interfaces such as the Gemini API within educational systems, and leveraging them to automate certain educational tasks that previously relied entirely on human intervention.

8.8 Aspects of Innovation

The proposed system distinguishes itself from traditional learning management systems by adding intelligent capabilities aimed at enhancing the user experience, as the role of the system is no longer limited to storing and displaying educational content, but has become capable of analyzing content and providing assistive educational services.

The most notable aspects of innovation are represented in:

- Integrating an intelligent assistant that helps students understand content and answer questions related to the course.
- Automating the process of quiz creation based on educational files uploaded by the instructor.
- Improving interaction with educational content through the use of artificial intelligence technologies.
- Building the system using modern software technologies that allow for scalability and the addition of future functions.

8.9 Evaluation of Results and System Robustness

The evaluation results demonstrated that the system possesses a good level of performance and stability within the scope of the tests conducted, as it succeeded in operating the core functions and achieving the main objectives of the project. The practical tests also showed the system's ability to provide an effective user experience by reducing the time required to perform certain educational tasks and improving the way information is accessed.

Nevertheless, the current results represent an initial stage within the scope of an academic project, and cannot be considered a final evaluation of the system's performance on a large scale, as the system requires future testing involving a larger number of users and different operating environments to obtain a more comprehensive evaluation.

8.10 Challenges and Difficulties

During the analysis, design, and implementation phases, the project faced a number of technical and organizational challenges, the most notable of which are:

- Integrating external artificial intelligence services with the system and ensuring their compatibility with project requirements.
- Improving the quality of outputs generated by the artificial intelligence model and controlling the format of results.
- Dealing with limited time resources within the project implementation period.
- Testing the system and verifying the stability of its functions across various scenarios.
- Achieving a balance between adding intelligent features and maintaining the simplicity and ease of use of the system.

Addressing these challenges contributed to improving the final design of the system and gaining practical experience in developing modern intelligent systems.

8.11 Proposed Future Work

Despite achieving the project's core objectives, there are several avenues through which the system could be further developed in the future, including:

- Developing an intelligent recommendation system that suggests appropriate educational content for each student based on their performance and interests.
- Adding advanced learning analytics to track student progress and identify their weaknesses.
- Improving the capabilities of the intelligent assistant so that it becomes more capable of understanding long-term educational context.
- Supporting additional types of intelligent questions and quizzes.
- Adding automatic evaluation capabilities for textual answers.
- Conducting extensive usability tests involving a larger number of students and instructors.

8.12 Recommendations

Based on the project results, it is recommended to continue developing intelligent learning management systems and to leverage the rapid advancement in artificial intelligence technologies to improve the quality of e-learning.

It is also recommended that educational institutions wishing to adopt such systems pay attention to data security and user privacy, and ensure that artificial intelligence is used as an assistive tool to enhance the role of the teacher and student rather than as a replacement for them.

In conclusion, this project represents a practical experience in the field of integrating artificial intelligence with educational systems, and demonstrates the great potential that can be achieved when employing modern technologies to build smarter, more interactive, and more adaptable learning environments that meet future needs.

References

[1] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877-1901, 2020.

[2] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[3] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv:2307.09288, 2023.

[4] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 24824-24837, 2022.

[5] P. Liu et al., "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," ACM Computing Surveys, vol. 55, no. 9, pp. 1-35, 2023.

[6] S. Bubeck et al., "Sparks of Artificial General Intelligence: Early Experiments with GPT-4," arXiv:2303.12712, 2023.

[7] H. Chase, "LangChain Documentation," 2024. [Online]. Available: https://python.langchain.com/docs

[8] Meta AI, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," 2024. [Online]. Available: https://github.com/facebookresearch/faiss

[9] Pinecone Systems Inc., "Pinecone Documentation," 2024. [Online]. Available: https://docs.pinecone.io/

[10] Weaviate B.V., "Weaviate Documentation," 2024. [Online]. Available: https://weaviate.io/developers/weaviate

[11] Milvus Project, "Milvus Documentation," 2024. [Online]. Available: https://milvus.io/docs

[12] Apache Software Foundation, "Apache Lucene," 2024. [Online]. Available: https://lucene.apache.org/

[13] Elastic N.V., "Elasticsearch Guide," 2024. [Online]. Available: https://www.elastic.co/guide/

[14] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

[15] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.

[16] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.

[17] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in Proc. EMNLP, Doha, Qatar, pp. 1532-1543, 2014.

[18] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed., Draft. Stanford University, 2023.

[19] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[20] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[21] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.

[22] K. P. Murphy, Machine Learning: A Probabilistic Perspective. Cambridge, MA, USA: MIT Press, 2012.

[23] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.

[24] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.

[25] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[26] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2017.

[27] M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[28] G. Booch, Object-Oriented Analysis and Design with Applications, 3rd ed. Boston, MA, USA: Addison-Wesley, 2007.

[29] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

[30] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2003.

[31] React Team, "React Documentation," 2024. [Online]. Available: https://react.dev/

[32] Tailwind Labs, "Tailwind CSS Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

[33] Microsoft, "TypeScript Documentation," 2024. [Online]. Available: https://www.typescriptlang.org/docs/

[34] Node.js Foundation, "Node.js Documentation," 2024. [Online]. Available: https://nodejs.org/docs/latest/api/

[35] Chroma, "Chroma Documentation," 2024. [Online]. Available: https://docs.trychroma.com/

[36] MongoDB Inc., "MongoDB Manual," 2024. [Online]. Available: https://www.mongodb.com/docs/manual/

[37] Google, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/gemini-api/docs

[38] Vercel Inc., "Next.js Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[39] Auth.js Team, "Auth.js Documentation," 2024. [Online]. Available: https://authjs.dev/

[40] Zod Contributors, "Zod Documentation," 2024. [Online]. Available: https://zod.dev/

[41] Mongoose, "Mongoose ODM Documentation," 2024. [Online]. Available: https://mongoosejs.com/docs/

[42] Vercel Inc., "NextAuth.js Documentation," 2024. [Online]. Available: https://next-auth.js.org/